

Tangent-Space Methods for Nonlinear Control and Biomechanics

Volume IV: Human Motor Control
The Neural Architecture of Movement

Preface

Draft Notice. This volume is under active development. Chapter depth and validation vary; some sections remain preliminary. The biology and nonlinear-dynamics chapter includes worked examples and exercises with independently checked mechanics. That review does not certify every chapter, a neural mechanism, or a human performance model. Source and validation records distinguish the completed corrections from the remaining review.

Volumes 0–III developed the mathematical and physical foundations for understanding motion: differential geometry, nonlinear control, trajectory optimization, and biomechanics. This volume asks a fundamentally different question: *how does the brain actually control the body?*

The golfer coordinates a body, a flexible implement and changing contacts using actuators whose force capacity depends on activation and mechanical history. The relevant model dimension, movement duration, precision and feedback latency depend on the task and measurement; they cannot be reduced to one universal count or timing budget.

This volume investigates how parallel processing, recurrent feedback, prediction, learned policies and intrinsic mechanics can cooperate. Cortical histological layers are not sequential computational layers, and a synaptic delay divided into a movement duration does not establish network depth or prove one neural architecture. A low-dimensional or shallow controller can be a useful engineering construction without uniquely explaining human control.

The purpose is to connect explicit dynamical models with observations that could support or challenge them. For golf and humanoid manipulation alike, the questions concern available control authority, coupled stability, energy and task accuracy under stated physical and informational constraints.

Contents

Preface	i
Nomenclature	v
1 The Degrees-of-Freedom Problem	1
1.1 Bernstein’s Problem as a Modeling Question	1
1.1.1 Configuration Freedom and Muscle Allocation	1
1.1.2 What Is the Task in a Golf Swing?	2
1.2 From Redundancy to Motor Abundance	2
1.3 The Uncontrolled Manifold Hypothesis	3
1.3.1 Coordinates, Metrics and Numerical Rank	4
1.3.2 Variance Is a Statistic, Not a Mechanism	4
1.4 A Reproducible Reaching Example	5
1.5 Learning, Constraints and Passive Dynamics	6
2 The Curse of Dimensionality — And How Humans Solve It	8
2.1 Classical Curse of Dimensionality	8
2.2 Biological Solutions	8
2.2.1 1. Synergy-Based Compression	9
2.2.2 2. Hierarchical Decomposition	9
2.2.3 3. Passive Dynamics Exploitation	9
2.2.4 4. Internal Models and Prediction	9
2.2.5 5. Task-Space Control	9
3 Neural Architecture of Motor Control	11
3.1 Cortical Motor Areas	11
3.1.1 Primary Motor Cortex (M1)	12
3.1.2 Premotor Cortex (PMC) and Supplementary Motor Area (SMA)	12
3.2 Subcortical Motor Structures	12
3.2.1 Cerebellum: The Forward Model	12
3.2.2 Basal Ganglia: Action Selection	13
3.3 Spinal Cord: The Local Controller	13
4 Shallow-but-Wide: Latency and State	15
4.1 A Conditional Argument About Speed	15
4.1.1 Define the Clock Before Counting Stages	15
4.2 Anatomical Layers and Computational Paths	16
4.3 What a Network Comparison Can Establish	17
4.3.1 Approximation Is Not a Control Guarantee	18
4.3.2 Hierarchy and Information Loss	18
4.4 Synergies: Reconstruction and Interpretation	19
4.4.1 A Reproducible Factorization Example	19
4.5 From This Hypothesis to a Golf Experiment	21

5	Ideomotor Theory and the Predictive Brain	23
5.1	Historical Ideomotor Theory	23
5.2	The Predictive Processing Framework	24
5.2.1	Active Inference	24
5.3	Connection to Control Theory	25
6	Internal Models: Forward and Inverse	26
6.1	Forward Models	26
6.1.1	The MOSAIC Architecture	27
6.2	Inverse Models	28
6.3	Smith Predictor for Neural Delays	28
7	Passive Distributed Control	29
7.1	The Frans Bosch Framework	29
7.2	Passive Dynamic Walking	30
7.3	Passive Dynamics in the Golf Swing	31
7.4	Bernstein's Stage 3: Exploiting DOF	31
8	Biology and Nonlinear Dynamics	33
8.1	Introduction: Mechanics With a Biological State	33
8.1.1	Bernstein, Constraints and the Scope of the Claim	33
8.2	Muscle, Tendon and Activation Dynamics	34
8.2.1	A Consistent Hill-Type Model	34
8.2.2	Length and Velocity Conventions That Can Be Checked	35
8.2.3	Zero Excitation Retains the Activation History	35
8.2.4	Preflex Response and Tendon Energy	36
8.3	Attractors and Coordination Transitions	36
8.3.1	What Stability Does and Does Not Establish	36
8.3.2	A Cost Crossing Is Not a Dynamical Bifurcation	37
8.3.3	The Haken–Kelso–Bunz Model With Its Correct Transition	37
8.4	Synergies, Task Geometry and Control Authority	38
8.4.1	What a Low-Dimensional Fit Establishes	38
8.4.2	A Task Level Set and Its Tangent Space	39
8.4.3	Instantaneous Rank Is Not Controllability	40
8.5	Impedance and Co-Contraction	40
8.5.1	Force, Displacement and Velocity Ports	40
8.5.2	How Muscle Forces Contribute to Joint Stiffness	41
8.5.3	An Explicit Optimization Example	41
8.5.4	Mechanical Work and Metabolic Cost	42
8.6	Pre-Stress and Stability of Interacting Subsystems	42
8.6.1	What a Tensegrity Analogy Can Contribute	42
8.6.2	Why Stable Parts Can Form an Unstable Whole	43
8.7	Building and Testing a Coupled Movement Model	43
8.7.1	Choose the Model for the Question	43
8.7.2	Identification Requires More Than a Motion Fit	44
8.7.3	A Golf and Humanoid Experiment Ladder	44
8.7.4	What Would Count Against the Proposed Explanation?	45
8.8	Connection to the Series	45

9	Central Pattern Generators	47
9.1	CPGs in Biological Motor Control	47
9.2	The Matsuoka Oscillator	48
9.3	Phase Coupling and Inter-Limb Coordination	49
10	Motor Learning and Adaptation	50
10.1	Three Learning Systems	50
10.2	Visuomotor Adaptation	50
10.3	Savings and Interference	52
10.3.1	Savings	52
10.3.2	Interference and Dual-Rate Learning	52
10.4	Structural Learning and Meta-Learning	53
11	Computational Models of Motor Control	54
11.1	Optimal Feedback Control	54
11.1.1	The Minimum Intervention Principle	55
11.2	Signal-Dependent Noise	56
11.3	Active Inference (Friston Framework)	57
12	From Neural Architecture to Robot Control	58
12.1	Lessons From Biology for Robot Design	58
12.2	Cerebellar-Inspired Adaptive Control	59
12.3	CPG-Based Locomotion Controllers	59
12.4	Whole-Body Control Architectures	61
12.5	Companion Software: Practical Implementation	61

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

These are default conventions; each chapter declares its model, frames, and any local reuse of symbols. The mechanical entries below use n independent, regular local configuration coordinates and $\mathbf{v} = \dot{\mathbf{q}}$. This convention is local: it does not identify every configuration manifold with a Euclidean space or apply unchanged to redundant representations.

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Configuration coordinates in the declared local chart.

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Coordinate rates; the generalized velocities under the local convention above.

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Second time derivatives of these coordinates, not necessarily Cartesian point accelerations.

$\mathbf{x} \in \mathbb{R}^{n_x}$ State-vector representation with n_x components; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ locally, with actuator or internal states added when needed. Redundant state representations also retain their constraints.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces conjugate to $\mathbf{q}$, with power $\boldsymbol{\tau}^T \dot{\mathbf{q}}$; components need not all be joint torques.

$t \in \mathbb{R}$ Time.

More generally, write $\mathbf{q} \in \mathbb{R}^{n_q}$ for a configuration representation and $\mathbf{v} \in \mathbb{R}^{n_v}$ for generalized velocities, with $\dot{\mathbf{q}} = \mathbf{N}(\mathbf{q})\mathbf{v}$ on the admissible domain. A unit quaternion has four components with a norm constraint, while angular velocity has three components in a declared frame. Configuration rates and generalized velocities therefore need not have the same dimension. See [Tedrake's multibody notes](#) for this representation convention. Constraints and any time-dependent kinematics must be included in the chosen model.

Inertia and Dynamics

M(**q**) Mass (inertia) matrix. It is symmetric positive definite for independent regular velocities with strictly positive modeled kinetic energy for every nonzero velocity. Constrained, redundant, massless, or singular representations require their own treatment.

C(**q**, $\dot{\mathbf{q}}$) Coriolis/centrifugal matrix; **C** $\dot{\mathbf{q}}$ is the generalized velocity-product force in the local mechanical equation. The matrix factorization is not unique.

g(**q**) Generalized gravity term; its sign follows the side of the mechanical equation on which the chapter places it.

J(**q**) Coordinate Jacobian of a time-independent task map **p**(**q**), so $\dot{\mathbf{p}} = \mathbf{J}\dot{\mathbf{q}}$ in the stated coordinates and frames.

With $\dot{\mathbf{q}} = \mathbf{N}\mathbf{v}$, the corresponding velocity Jacobian is $\mathbf{J}_v = \mathbf{J}_q\mathbf{N}$. Force components must change with the velocity convention: $\boldsymbol{\tau}_v = \mathbf{N}^T\boldsymbol{\tau}_q$ gives $\boldsymbol{\tau}_v^T\mathbf{v} = \boldsymbol{\tau}_q^T\dot{\mathbf{q}}$. Thus the same club-point motion and mechanical power must result from consistently paired kinematic and force representations.

Control and Optimization

For the indexed entries, declare a discrete update $\mathbf{x}_{k+1} = F_k(\mathbf{x}_k, \mathbf{u}_k)$ and a nominal pair $(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$. Its derivatives are evaluated at that pair. These are derivatives of the full update, including the chosen sampling or numerical integration rule, not of zero-input drift alone.

A_k = $D_x F_k$ State Jacobian of the declared discrete update at the nominal pair.

B_k = $D_u F_k$ Input Jacobian of that update at the same pair.

V(**x**) or $\mathcal{V}$ Value function or Lyapunov function.

Q State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta\mathbf{u} = -\mathbf{K}\delta\mathbf{x}$).

γ A curve with a declared parameter; for specified initial data and an input law, $\gamma(t)$ denotes a particular timed state trajectory. A solution map $\phi(t; t_0, \mathbf{x}_0)$ instead retains dependence on initial time and state, for a fixed evolution law on its domain of existence.

Continuous-time chapters define their Jacobians from the full differential equation. Some use $f(\mathbf{x}, \mathbf{u}, t)$ for that full right-hand side; the autonomous $f(\mathbf{x})$ below denotes only the declared zero-input drift. Neither convention makes a discrete input Jacobian equal to the continuous one without accounting for the sampling interval and evolution.

Affine Control and Counterfactuals

$f(\mathbf{x})$ Drift vector field: complete autonomous evolution of the declared effective plant when $\mathbf{u} = \mathbf{0}$.

$G(\mathbf{x})$ Input (control) matrix field mapping control inputs to state derivatives.

$\dot{\mathbf{x}} = f(\mathbf{x}) + G(\mathbf{x})\mathbf{u}$ Control-affine system equations of motion.

$\mathbf{x}^{\text{ZTCF}}(t)$ Forward Zero-Torque Counterfactual trajectory under $\mathbf{u}(t) \equiv \mathbf{0}$.

$\mathbf{a}_{\text{ZVCF}}(\mathbf{q}, \mathbf{z})$ Instantaneous Zero-Velocity Counterfactual acceleration with velocity and declared control set to zero.

$\text{DCR}_{W,\mathcal{U}}(\mathbf{x})$ Drift-Control Ratio in a declared acceleration or task-projected space:

$$\text{DCR}_{W,\mathcal{U}}(\mathbf{x}) = \frac{\|W\mathbf{a}_{\text{drift}}\|_2}{\sup_{\mathbf{u} \in \mathcal{U}} \|WB_a\mathbf{u}\|_2 + \varepsilon}.$$

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F}_{\text{muscle}}$ Force generated by muscles.

$\mathbf{W}$ Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

The Degrees-of-Freedom Problem

A successful swing is an outcome of coordinated motion, forces and contact. It does not specify a unique joint trajectory or muscle history. The question associated with Bernstein's account of coordination [5] is how a body with many available motions produces a repeatable task outcome while remaining adaptable. Answering it requires distinguishing three things: the geometry of equivalent outcomes, the dynamics that make a movement feasible, and the evidence from which a controller might be inferred.

1.1 Bernstein's Problem as a Modeling Question

Definition 1.1. The Degrees-of-Freedom Problem

Let $\mathcal{Q}$ be an n -dimensional configuration manifold and let a local chart be denoted by $\mathbf{q}$. Choose a task map $h : \mathcal{Q} \rightarrow \mathbb{R}^k$ and a desired value $\mathbf{y}_0$. The configurations satisfying that task form the level set $\mathcal{M}_{\mathbf{y}_0} = \{\mathbf{q} : h(\mathbf{q}) = \mathbf{y}_0\}$. Where the task Jacobian has constant rank r in a neighborhood, the local level-set dimension is $n - r$; it is $n - k$ only for k independent task constraints. This counts local configuration freedom, not feasible control histories.

A seven-coordinate arm with a regular three-dimensional endpoint-position map has four local task-null directions. If a locally independent six-dimensional pose is specified instead, it has one. These are illustrative model counts, not anatomical measurements. Dependent constraints must not be counted twice, and joint limits can remove available motion even when a Jacobian has a nontrivial kernel. At a singular configuration, rank can change and a tangent-kernel count alone need not describe the dimension of an exact solution set.

1.1.1 Configuration Freedom and Muscle Allocation

At a fixed posture, a simplified muscle-force allocation model writes

$$R(\mathbf{q})\mathbf{F} = \boldsymbol{\tau}_{\text{req}}, \quad 0 \leq F_i \leq F_{i,\text{max}}, \quad (1.1)$$

where $R \in \mathbb{R}^{n \times p}$ is a signed moment-arm matrix, $\mathbf{F}$ contains p tensile forces, and $\boldsymbol{\tau}_{\text{req}}$ is the required generalized load after other modeled contributions have been separated. If the equality is feasible, its unconstrained affine solution space has dimension $p - \text{rank } R$, not automatically $p - n$. Force limits restrict it to a feasible set that may contain a continuum, one point, or no solution. Activations and excitations are distinct variables: force also depends on muscle length, velocity, tendon state and history.

For example, use a dimensionless toy map

$$R = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \quad \tau_{\text{req}} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}.$$

The equality gives $\mathbf{F} = (3 - t, 3 - t, t)^T$. Bounds $0 \leq F_i \leq 3$ leave $0 \leq t \leq 3$; imposing $F_{3,\text{max}} = 0$ leaves only $(3, 3, 0)^T$. A nullity of one therefore does not guarantee compensatory capacity. Configuration and force nullspaces act on different physical variables; adding their dimensions does not generally yield a trajectory redundancy. The familiar arithmetic $p - k$ can describe a particular regular algebraic model with $n + p$ variables and $n + k$ independent equalities, but those assumptions must be established rather than inferred from anatomical counts.

A movement must additionally satisfy equations of motion, contact constraints, actuator dynamics and boundary conditions. A kinematically equivalent posture can change inertial coupling, moment arms, force capacity, energy use and the response to the next perturbation. This connects redundancy to the muscle-state and load-accounting treatment in Volume III rather than providing an alternative to it.

1.1.2 What Is the Task in a Golf Swing?

Six pose coordinates describe a local rigid-club position and orientation. They do not specify clubhead velocity, the face contact point, collision conditions or the resulting ball flight. For a swing outcome, a more useful terminal map can depend on configuration, velocity and implement state: $\mathbf{y} = g(\mathbf{x}(T))$. The state $\mathbf{x}$ may include shaft deformation and internal actuator states; T and the definition of impact must also be specified. A Jacobian of this state map has a different domain from $J_h(\mathbf{q})$.

Consequently, there is no universal “200 coordinates minus six constraints” calculation for golf, nor a single neural delay that determines the available controller dimension. Model resolution, task tolerance, contact mode and measurement bandwidth determine what a chosen count means. A small variation in club position can coexist with a consequential change in delivery velocity or face orientation. Coordination should be evaluated against the declared outcome and its uncertainty, not solely against visual similarity of joint motion.

1.2 From Redundancy to Motor Abundance

Key Insight 1.1. Redundancy as Motor Abundance

Extra feasible solutions can provide alternatives when conditions change. They may allow a golfer to preserve an outcome while changing joint loading, or a humanoid to avoid an obstacle while maintaining a grasp. This is an opportunity conditional on available force, motion and information; a high-dimensional model alone guarantees neither robustness nor learning.

Latash’s abundance perspective emphasizes families of useful solutions rather than the elimination of every additional degree of freedom [27]. That interpretation motivates examining task-preserving covariation. It is not a proof of one neural implementation or a rejection of every optimization model: a feedback policy can also permit many realizations.

Variability that is irrelevant to one output may matter to fatigue, contact stability, another objective, or a later time.

1.3 The Uncontrolled Manifold Hypothesis

The UCM approach compares observed variability with directions that locally preserve a selected task variable [36]. Its original sit-to-stand study tested alternative task hypotheses in nine participants; that evidence does not establish a universal golf controller. Here “uncontrolled” denotes a proposed relative pattern of stabilization, not an uncontrollable subspace of a dynamical system. Reachability concerns which states inputs can attain; a task kernel concerns which instantaneous variations an output fails to distinguish.

Definition 1.2. The Uncontrolled Manifold

Distinguish the exact level set $\mathcal{M}_{\mathbf{y}_0}$ from its local linear approximation. At a regular point $\mathbf{q}_0$ with $h(\mathbf{q}_0) = \mathbf{y}_0$,

$$T_{\mathbf{q}_0}\mathcal{M}_{\mathbf{y}_0} = \ker J_h(\mathbf{q}_0) = \{\delta\mathbf{q} : J_h(\mathbf{q}_0)\delta\mathbf{q} = \mathbf{0}\}. \quad (1.2)$$

The kernel is commonly called the linearized UCM. It preserves the task to first order; it is not generally the exact curved set of finite task-preserving displacements.

Taylor expansion makes the limitation explicit:

$$h(\mathbf{q}_0 + \delta\mathbf{q}) - h(\mathbf{q}_0) = J_h\delta\mathbf{q} + \frac{1}{2}D^2h[\delta\mathbf{q}, \delta\mathbf{q}] + \mathcal{O}(\|\delta\mathbf{q}\|^3). \quad (1.3)$$

This expression assumes sufficient local smoothness and a common coordinate chart. A null first-order term leaves the curvature term. For $h(q_1, q_2) = q_1^2 + q_2^2$ at $(1, 0)$, $(0, \epsilon)$ is tangent but gives $h(1, \epsilon) = 1 + \epsilon^2$. At $\epsilon = 0.1$ the task changes by 0.01; the exact nearby solution is $(\sqrt{1 - \epsilon^2}, \epsilon)$. At the singular point $(0, 0)$, by contrast, $J_h = 0$ while the zero level set contains only that point. A two-dimensional kernel is not a two-dimensional family of exact zero-task solutions there.

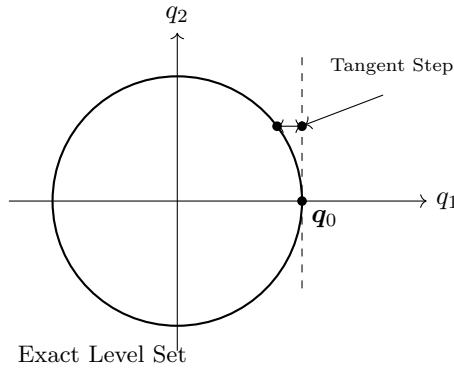


Figure 1.1: Tangent Steps and Exact Task Preservation for a Curved Level Set

In Plain Language 1.1. Mathematical Reminder: The Geometry of the UCM

A tangent gives the direction of a road at one point, not the road's whole path. Following the tangent in a straight line eventually leaves a curved road. Similarly, tangent-aligned variations can still change a nonlinear task. Maintaining the exact task requires following the level set or correcting its finite error; a covariance ellipse alone does not provide that mechanism.

1.3.1 Coordinates, Metrics and Numerical Rank

Choose a chart and coordinate scales before saying that two directions are orthogonal. With positive diagonal scales D , analyze $\delta\mathbf{z} = D^{-1}\delta\mathbf{q}$ and $J_z = J_h D$. Euclidean orthogonality in $\mathbf{z}$ corresponds to the metric $D^{-T}D^{-1}$ in $\mathbf{q}$. Mixing metres and radians without a declared scale makes a variance ratio ambiguous; even changing one coordinate's scale changes that metric. Joint angles crossing a branch cut must be unwrapped or treated in an appropriate local chart before taking an arithmetic mean.

For the chosen scaled Jacobian, an SVD supplies orthonormal bases N for $\ker J_z$ and B for $\text{range}(J_z^T)$. The projectors are $P_N = NN^T$ and $P_B = BB^T$, with $P_N + P_B = I$. Numerical rank depends on a stated singular-value tolerance. The implementation below uses a relative tolerance, so multiplying the whole Jacobian by a nonzero scalar does not change rank in ordinary finite precision. For heterogeneous outputs, task scaling and near-singular directions still require an explicit sensitivity analysis.

1.3.2 Variance Is a Statistic, Not a Mechanism

Let $N_s \geq 2$ repeated observations be aligned at a declared event or phase, with scaled deviations $\mathbf{d}_i = \mathbf{z}_i - \bar{\mathbf{z}}$. Evaluate the task Jacobian at the corresponding mean configuration. Using sample covariance $C = (N_s - 1)^{-1} \sum_i \mathbf{d}_i \mathbf{d}_i^T$, define

$$V_N = \frac{\text{tr}(P_N C)}{n - r}, \quad V_B = \frac{\text{tr}(P_B C)}{r}, \quad \rho = \frac{V_N}{V_B}. \quad (1.4)$$

These expressions require $0 < r < n$ and $V_B > 0$ for a finite ratio. A zero-dimensional group has no per-dimension variance; $0/0$ is not evidence of perfect stabilization. The function returns `None` for these undefined quantities. The $N_s - 1$ covariance convention is unbiased for independent, identically distributed observations with finite covariance; it does not make correlated samples, pooled participants or phase-misaligned swings independent trials.

A value $\rho > 1$ describes greater variance per dimension along the chosen linearized task-null directions. Isotropic covariance gives $\rho = 1$ when both groups exist. A sample ratio needs uncertainty analysis and appropriate comparisons; it is not a stand-alone significance test. Also, the complementary variance does not equal task-output variance: to first order the latter is $J_z C J_z^T$, weighted by task sensitivities. Changing phase alignment, coordinate scales, measurement errors or the task map can change the interpretation.

A manufactured counterexample demonstrates the inference limit. For a stable linear stochastic model $d\mathbf{z} = A\mathbf{z} dt + L d\mathbf{W}$, let $\mathbf{W}$ be standard Brownian motion and write $Q = LL^T$. Stationary covariance satisfies $AC + CA^T + Q = 0$. Both

$$C = \text{diag}(1, 9), \quad (A_1, Q_1) = (-I, 2C), \quad (A_2, Q_2) = (-\text{diag}(9, 1), 18I)$$

satisfy this equation. For task Jacobian $(1,0)$, both give $\rho = 9$. The first has isotropic restoring dynamics and anisotropic noise; the second has directional restoring dynamics and isotropic noise. Thus identical covariance cannot identify which mechanism generated it. Controlled perturbations, temporal information and independent noise models can help discriminate hypotheses. This algebra is not a fitted neural model.

1.4 A Reproducible Reaching Example

Run this listing from the repository root with its pinned Python dependencies. The tested implementation in `src/affine_control/ucm_analysis.py` validates array shape, finite real values, rank tolerance and numerical range. Its input is a fixed Jacobian, so the caller owns the task model and its linearization point. The three-link arm below uses relative joint angles in radians and link lengths in metres, with Euclidean joint-angle scaling.

```

1 import numpy as np
2 from src.affine_control.ucm_analysis import ucm_variance
3
4
5 def planar_endpoint(angles, lengths):
6     headings = np.cumsum(angles, axis=-1)
7     x = np.sum(lengths * np.cos(headings), axis=-1)
8     y = np.sum(lengths * np.sin(headings), axis=-1)
9     return np.stack((x, y), axis=-1)
10
11
12 def planar_jacobian(angles, lengths):
13     headings = np.cumsum(angles)
14     terms = np.array([-lengths * np.sin(headings),
15                      lengths * np.cos(headings)])
16     return np.cumsum(terms[:, ::-1], axis=1)[:, ::-1]
17
18
19 nominal = np.array([0.5, 1.2, -0.3])
20 lengths = np.array([0.3, 0.25, 0.2])
21 jacobian = planar_jacobian(nominal, lengths)
22 _, _, vt = np.linalg.svd(jacobian, full_matrices=True)
23 basis = np.column_stack((vt[2:].T, vt[:2].T))
24 # Six symmetric observations: sample mean = nominal.
25 # N-1 = 5; paired axes have covariance diag(sigma**2).
26 sigma = np.array([0.03, 0.01, 0.01])
27 axes = np.sqrt(2.5) * np.diag(sigma)
28 deviations = np.vstack((axes, -axes)) @ basis.T
29 result = ucm_variance(nominal + deviations, jacobian)
30 iso_axes = np.sqrt(2.5) * 0.01 * np.eye(3)
31 iso_data = nominal + np.vstack((iso_axes, -iso_axes))
32 isotropic = ucm_variance(iso_data, jacobian)
33 step = 0.03 * vt[2]
34 null_step_linear_error = np.linalg.norm(jacobian @ step)
35 null_step_actual_error = np.linalg.norm(
36     planar_endpoint(nominal + step, lengths)
37     - planar_endpoint(nominal, lengths))
38 print(result)          # VN = 0.0009, VB = 0.0001, ratio = 9
39 print(isotropic.ratio) # 1, apart from roundoff
40 print(null_step_actual_error) # nonzero despite J @ step ~ 0

```

Listing 1.1: Manufactured Reaching Variance and Finite Task Error

The larger null variance is deliberately constructed, not discovered in experimental data. Six deterministic points demonstrate covariance accounting; they do not estimate human variability or provide a statistical confidence interval. The finite endpoint residual exposes the linearization limit. The tests compare the printed Jacobian with Cartesian finite differences and verify both the anisotropic example and its isotropic control.

1.5 Learning, Constraints and Passive Dynamics

Freezing, freeing and exploiting degrees of freedom are useful questions to bring to descriptions of skill acquisition associated with Bernstein's framework [5]; they should not be imposed as a universal three-stage law or a fixed sequence for every golfer. A joint moving through a small range is not thereby absent from the model. True mechanical locking, increased stiffness, correlated motion and a changed control policy have different effects on the system's feasible states. None implies that configuration dimension becomes exactly the task dimension.

Likewise, wrist release is a coupled dynamical event involving geometry, applied loads and joint/implement mechanics; naming a centrifugal effect does not establish zero muscle action or a unique expert mechanism. A control-affine model $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{G}(\mathbf{x})\mathbf{u}$ can describe the interaction of drift and inputs under stated assumptions. Reachability depends on that complete system and its admissible controls [1]. Removing a physical constraint can add feasible trajectories when the old ones remain admissible in the enlarged model, but merely observing more joint motion proves neither a larger reachable set nor better task performance.

For golf and humanoid movement, the useful chain is therefore explicit: choose the outcome; identify its local geometry; impose the coupled physical constraints; then test candidate explanations against repeated observations and perturbations. This preserves the insight of motor abundance while keeping kinematic equivalence, dynamic feasibility and neural inference distinct.

Exercises

1. **Rank and Feasibility.** Compute the rank and kernel of $J = \begin{bmatrix} 1 & 0 \\ 2 & 0 \end{bmatrix}$. Compare its nullity with the force-allocation example and impose $F_{3,\max} = 0$. Explain why equal nullity does not imply equal physical freedom.
2. **Curvature.** For the circle task, compare a tangent step with the exact level-set point at $\epsilon = 0.05$ and 0.1 . Verify the second-order task residual and discuss the singular point at the origin.
3. **Variance and Scaling.** Execute the reaching example. Change the coordinate scales consistently in both data and Jacobian; explain why this changes the metric and can change the variance ratio. Test rank-zero, full-rank and zero-variance inputs and interpret **None**.
4. **Identifiability.** Verify both stationary-covariance equations. Propose a perturbation or time-lag measurement that could distinguish the models despite their identical instantaneous covariance.
5. **Golf Task Design.** Define a delivery outcome including the measurement reference point, event time and uncertainty. Compare it with a position-only task. State which additional data would be needed to distinguish kinematic compensation from a particular muscle or neural mechanism.

Chapter 2

The Curse of Dimensionality — And How Humans Solve It

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

In ten dimensions, the unit hypercube has $2^{10} = 1024$ corners. In a hundred, it has $2^{100} \approx 10^{30}$. The nervous system controls hundreds of dimensions every second.

2.1 Classical Curse of Dimensionality

The curse of dimensionality [4] states that the computational resources needed to solve many problems grow *exponentially* with the state dimension n :

Definition 2.1. The Curse of Dimensionality

For a control problem with state dimension n :

- **Grid-based methods** (dynamic programming): cost $\sim \mathcal{O}(N^n)$ where N is the grid resolution per dimension
- **Sample-based methods** (Monte Carlo, RL): sample complexity grows polynomially in n but with large constants
- **Trajectory optimization** (DDP, collocation): cost $\sim \mathcal{O}(n^3 \cdot T)$ per iteration — the only scalable approach for deterministic systems

2.2 Biological Solutions

Humans solve the dimensionality problem through five complementary mechanisms:

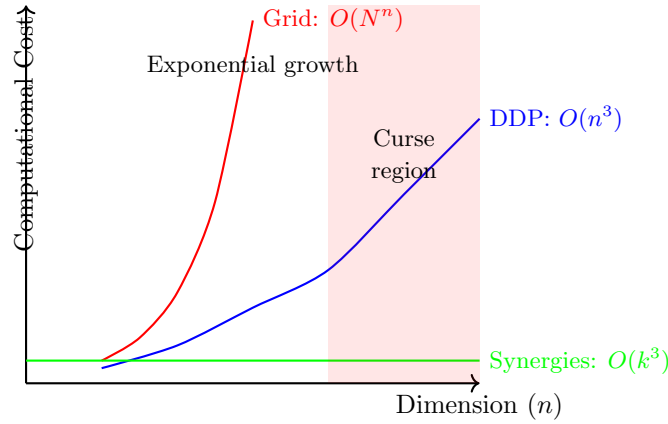


Figure 2.1: The curse of dimensionality: computational cost grows exponentially with dimension for grid-based methods, cubically for DDP, and remains constant when using synergy-based dimensionality reduction (where $k \ll n$).

2.2.1 1. Synergy-Based Compression

As introduced in Chapter 4: $\mathbf{u}(t) = W\mathbf{c}(t)$ reduces p -dimensional muscle space to k -dimensional synergy space, with $k/p \approx 4/400 = 1\%$.

2.2.2 2. Hierarchical Decomposition

Multi-level processing decomposes the global problem into independent subproblems.

2.2.3 3. Passive Dynamics Exploitation

The drift field $f(\mathbf{x})$ does useful work. The controller only corrects deviations. This reduces the effective control dimension (Volume II, passive dynamics).

2.2.4 4. Internal Models and Prediction

Forward models predict consequences, enabling open-loop execution with sparse feedback corrections (see Chapter 6).

2.2.5 5. Task-Space Control

Control in a low-dimensional task space rather than the full joint/muscle space. The null space absorbs variability (UCM framework, Chapter 1).

```
1 import numpy as np
2
3 def computational_cost_table(n_dims: list[int]) -> None:
4     """Print computational cost for different methods vs dimension."""
5     print(f"{'n_dim':>6} {'Grid (N=10)':>15} {'DDP (T=100)':>15} "
6           f"{'Synergy (k=5)':>15}")
7     print("-" * 55)
8     for n in n_dims:
```

```

9     grid_cost = 10**n if n <= 10 else float('inf')
10    ddp_cost = n**3 * 100
11    synergy_cost = 5**3 * 100 # After reduction to k=5
12    print(f"{n:>6d} {grid_cost:>15.0e} {ddp_cost:>15.0e} "
13          f"{synergy_cost:>15.0e}")
14
15 computational_cost_table([2, 5, 10, 20, 50, 100, 200])

```

Listing 2.1: Computational cost comparison across methods.

Exercises

1. **Scaling Analysis.** Using the computational cost table, determine the maximum state dimension for which grid-based dynamic programming is feasible (assuming 10^{12} FLOPS).
2. **Synergy Efficiency.** If a motor task uses 100 muscles with 5 synergies, by what factor does the synergy decomposition reduce the dimensionality of the control problem?
3. **(Programming.)** Implement DDP for a 2-DOF, 5-DOF, and 10-DOF chain. Measure computation time per iteration. Verify cubic scaling in state dimension.

Chapter 3

Neural Architecture of Motor Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The motor system is not one controller. It is an orchestra of specialized modules, each playing its part in the symphony of movement.

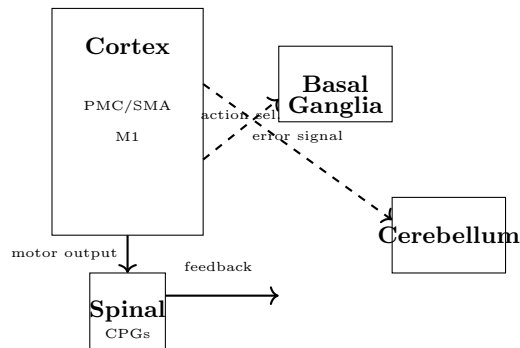


Figure 3.1: Hierarchical neural motor architecture: cortex plans/initiates, basal ganglia selects actions, cerebellum provides error-based learning, spinal cord implements CPGs and reflex circuits.

3.1 Cortical Motor Areas

The cerebral cortex contains several distinct motor areas, each with specialized roles:

3.1.1 Primary Motor Cortex (M1)

M1 (Brodmann area 4) is the final cortical output stage for voluntary movement. Layer V contains the giant pyramidal (Betz) cells whose axons form the corticospinal tract—the most direct neural pathway from cortex to spinal motor neurons.

Key properties of M1 (experimentally confirmed):

- **Somatotopic organization:** body parts are mapped to cortical regions (the motor homunculus), with overlapping representations [35]
- **Population coding:** individual neurons have broad cosine-shaped directional tuning about a preferred direction [12], and movement direction is recovered from the vector sum of the population's contributions rather than from any single cell [13]
- **Preparatory activity:** M1 neurons show activity 100–200 ms before movement onset during motor preparation, reflecting planning processes

3.1.2 Premotor Cortex (PMC) and Supplementary Motor Area (SMA)

- **PMC:** movement planning based on external cues (visually guided reaching)
- **SMA:** movement planning based on internal cues (self-initiated sequences)
- Both areas project to M1 and directly to the spinal cord

3.2 Subcortical Motor Structures

3.2.1 Cerebellum: The Forward Model

The human cerebellum contains roughly 69 billion neurons—about 80% of all the neurons in the brain, and overwhelmingly granule cells—against some 16 billion in the cerebral cortex [3]. Electrophysiological and lesion studies establish its primary role in motor control as a **forward model**:

Neural Architecture 3.1. The Cerebellum as a Forward Model

The cerebellar circuit [31, 2, 24]:

1. Receives a copy of the motor command (efference copy) via the pontine nuclei
2. Receives sensory feedback via the inferior olive and mossy fibers
3. Computes the predicted sensory consequences of the command using parallel fiber–Purkinje cell synapses (basis function expansion)
4. The climbing fiber signal carries sensory prediction error from the inferior olive
5. Error-driven learning updates Purkinje cell responses through long-term depression (LTD) at parallel fiber–Purkinje cell synapses. LTD is induced only when parallel fiber activity and the climbing fiber spike fall within a window of a few hundred milliseconds of each other; the optimum is region-specific and appears tuned to each region's own sensorimotor feedback delay [41]

In the language of control theory (Volume I):

- The cerebellum implements $\hat{\mathbf{x}}_{k+1} = f(\hat{\mathbf{x}}_k, \mathbf{u}_k)$ — the state prediction step of the Kalman filter
- The climbing fiber error signal implements the innovation update
- The granule cell layer provides the basis functions for the nonlinear mapping

Two lines of evidence support this architecture, and it is worth keeping them apart because they come from different populations.

In healthy subjects, reaching against a novel force field is initially distorted and then straightens with practice; removing the field produces aftereffects that mirror the original distortion, which is what implicates a learned internal model of the limb's dynamics rather than a stiffer or better-tuned feedback loop [38]. In patients with cerebellar degeneration, on-line correction within a movement is intact while trial-to-trial learning of those same altered dynamics is profoundly impaired—a dissociation that does not appear in Huntington's disease, where the deficit falls on the feedback side instead [39]. The first result establishes that an internal model is learned; the second locates the learning in the cerebellum.

Two timescales are easy to conflate here, and they are not the same quantity. The window in which parallel fiber and climbing fiber activity must coincide for LTD to be induced is a few hundred milliseconds—that is a coincidence-detection requirement, set by how long the error signal takes to arrive. The depression it produces, and the behavioural adaptation that follows, accumulate over minutes and tens of trials. Motor learning being slow is therefore not in tension with the plasticity rule being fast: the millisecond figure governs *whether* a synapse is eligible to change, the minutes figure governs *how much* it has changed.

3.2.2 Basal Ganglia: Action Selection

The basal ganglia implement an action selection mechanism through competitive inhibition. The direct pathway facilitates a desired action; the indirect pathway suppresses competing actions. This is analogous to a model predictive controller evaluating multiple candidate trajectories and selecting the best one.

3.3 Spinal Cord: The Local Controller

The spinal cord is not merely a relay station. It contains:

- **Motor neuron pools:** lower motor neurons that directly innervate muscles
- **Interneuron networks:** local circuits that implement reflexes, pattern generation, and coordination
- **Central pattern generators (CPGs):** oscillatory circuits for rhythmic movements (Chapter 8)

Exercises

1. **Population Vector.** Given neurons with preferred directions every 45° , compute the population vector for a movement at 30° . Plot individual neuron contributions.
2. **Cerebellar Forward Model.** Implement a simple forward model using a lookup table. Show how climbing fiber errors update the model over successive trials.
3. **(Research.)** Read Georgopoulos et al. (1986) and summarize the evidence for population coding in M1.

Chapter 4

Shallow-but-Wide: Latency and State

A fast correction depends on what must be computed now, what is already represented, and what the mechanics do before feedback arrives.

4.1 A Conditional Argument About Speed

A golf swing joins neural computation to a moving mechanical system. The controller must act on incomplete, delayed information while muscles, tendons, contacts and the club continue to evolve. Parallel processing can shorten some computational paths, and a learned policy can turn an existing state estimate into a command without solving an optimization problem from scratch. Neither observation proves that the brain has a universally shallow architecture. The useful question is: *which new information must influence which output before a specified deadline?*

4.1.1 Define the Clock Before Counting Stages

A response measured at EMG onset, a change in muscle force, and successful recovery of the club trajectory have different endpoints. A budget must state the stimulus or perturbation onset, sensory modality, preparation and output landmark. Include sensory transduction, propagation, processing and the relevant output delays only once. Electromechanical delay belongs in a force-response budget; it does not belong after an endpoint defined as EMG onset.

There is no universal 1 ms lower bound for chemical synaptic delay. Katz and Miledi reported minimum delays of 0.4–0.5 ms at 20°C, measured from the peak presynaptic inward current to the onset of the postsynaptic current at a neuromuscular junction [23]. Those landmarks and temperature matter: this is neither a whole sensorimotor-loop latency nor a universal cortical processing time. Action-potential width, receptor rise time, refractory period, axonal travel time and input-to-output latency are different measurements.

Definition 4.1. A Stipulated Serial Depth Budget

For a serial model with L new processing stages, define

$$T_{\text{required}} = T_{\text{fixed}} + \sum_{j=1}^L \tau_j. \quad (4.1)$$

T_{fixed} contains endpoint-specific delays excluded from the stage costs τ_j . If every stage is assigned the same positive cost τ and $T_{\text{available}} \geq T_{\text{fixed}}$, then

$$L_{\text{max}} = \left\lfloor \frac{T_{\text{available}} - T_{\text{fixed}}}{\tau} \right\rfloor.$$

If the available time is smaller than the fixed delay, even zero additional stages cannot meet this endpoint deadline. An illustrative budget of 50 ms, 20 ms fixed delay and 5 ms per stage permits six stages. These numbers define a toy calculation; they are not measured golf-response times.

For a computation with parallel branches, use the longest required dependency path, including communication and synchronization costs, rather than the sum of all operations. For recurrence, unroll a *finite correction interval* with a declared initial state. A feedback loop cannot be assigned a finite critical path by treating its cyclic graph as an acyclic feedforward network.

Throughput is not latency. A pipeline may produce a command every millisecond while each command still depends on sensory information acquired much earlier. Conversely, an existing recurrent state can support a rapid readout without replaying its entire history. Prediction can exploit predictable motion, but it cannot incorporate an unpredictable disturbance before information about that disturbance is available. Thus a fresh correction's latency and the time used to establish the current policy or state estimate must be distinguished.

4.2 Anatomical Layers and Computational Paths

Cortical laminae describe anatomy; they are not a count of sequential artificial network transformations. Parallel projections, recurrent connections and multiple input and output populations prevent a six-lamina description from specifying six serial computations. Nor does a short response identify every circuit participating in preparation, ongoing estimation or later correction.

Even the anatomical description needs regional and species qualifications. Yamawaki and colleagues identified a thin population at the layer 3/5A border in mouse forelimb M1 with layer-4-like connectivity: thalamic inputs, ascending local outputs to layers 2/3 and relatively weak long-range outputs [45]. This challenges a categorical absence of M1 layer 4; it does not establish uniform granular anatomy in all motor cortex or a fixed number of synapses in a human golf-control loop.

Key Insight 4.1. Architecture as a Testable Model

Parallel pathways, recurrent state, prediction and hierarchical organization are candidate ways to coordinate a delayed system. Their benefit depends on available information, connectivity, learning, energy, communication and the mechanical task. Neither total neuron count nor histological layer count specifies the latency or accuracy of a controller. Width also has costs; its computation and communication cannot be assumed free.

4.3 What a Network Comparison Can Establish

A static artificial neuron applies an activation function to a weighted sum. This is a useful mathematical model, not a complete description of a biological neuron's dendrites, spikes or temporal state. The following example compares *untrained feedforward mappings*, not successful motor controllers. Both have 20 inputs, ten signed abstract outputs, ReLU hidden activations and a linear output. Signed outputs are not bounded muscle activations.

The deep model has nine hidden layers of width 50 followed by ten outputs: ten affine maps and 21,960 weights and biases. A single hidden layer of width h followed by ten outputs has $31h + 10$ parameters. Choosing $h = 708$ gives 21,958, two fewer than the deep model. This is nearly equal parameter count, not equal expressiveness, training difficulty or task performance. Choosing width 2000 instead would give 62,010 parameters and would not match that budget.

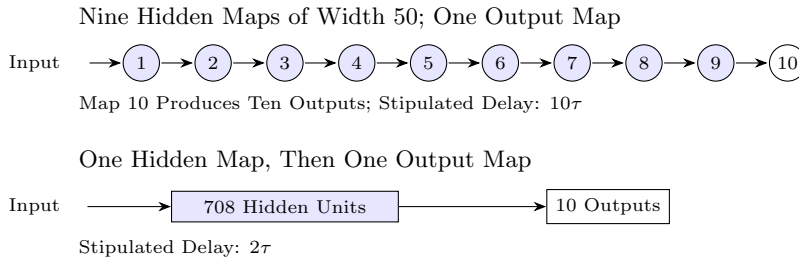


Figure 4.1: Counted Affine Maps Under an Equal Stage-Cost Assumption. The diagram shows computational stages, not cortical laminae or measured neural delays.

```

1 import numpy as np
2
3 Network = list[tuple[np.ndarray, np.ndarray]]
4
5
6 def make_network(widths: list[int], rng: np.random.Generator) -> Network:
7     """Create an untrained toy mapping with zero biases."""
8     return [(rng.normal(0, 0.1, (right, left)), np.zeros(right))
9             for left, right in zip(widths[:-1], widths[1:], strict=True)]
10
11
12 def parameter_count(model: Network) -> int:
13     """Count every weight and bias, including the output map."""
14     return sum(weights.size + bias.size for weights, bias in model)

```

```

15
16
17 def forward(model: Network, state: np.ndarray) -> np.ndarray:
18     """Use ReLU hidden maps and a common linear signed output."""
19     hidden = state
20     for weights, bias in model[:-1]:
21         hidden = np.maximum(0, weights @ hidden + bias)
22     weights, bias = model[-1]
23     return weights @ hidden + bias
24
25
26 rng = np.random.default_rng(42)
27 deep_model = make_network([20] + [50] * 9 + [10], rng)
28 wide_model = make_network([20, 708, 10], rng)
29 toy_state = rng.normal(size=20)
30 deep_output = forward(deep_model, toy_state)
31 wide_output = forward(wide_model, toy_state)
32 assumed_stage_ms = 5.0 # A declared model cost, not a timing measurement.
33 assumed_delays_ms = tuple(
34     len(model) * assumed_stage_ms for model in (deep_model, wide_model)
35 )
36 counts = (parameter_count(deep_model), parameter_count(wide_model))
37 assert counts == (21960, 21958)
38 assert assumed_delays_ms == (50.0, 10.0)

```

Listing 4.1: Parameter Counts and an Assumed Stage Delay

The fivefold delay ratio follows directly from assigning each affine map the same 5 ms cost, independent of width. It is not a measured software speedup or a biological result. Actual delays may change with width, hardware, memory movement and parallel resources. Random outputs establish no tracking accuracy; a meaningful comparison trains both controllers on a common task and tests held-out performance under measured input-to-output delays and actuator limits.

4.3.1 Approximation Is Not a Control Guarantee

Cybenko's theorem establishes uniform approximation of continuous functions on a compact unit cube by sufficiently many units with a continuous sigmoidal activation [10]. It does not supply a small required width, a successful training algorithm or closed-loop stability. That specific theorem also does not analyze the ReLU listing above. Approximating a state-to-command map on a bounded domain is different from reconstructing unavailable state or controlling trajectories that leave that domain. An approximate policy can perform poorly near constraints even when a global average error is small.

4.3.2 Hierarchy and Information Loss

Suppose a model deliberately assigns dimensions $n_l = n_0/r^l$, $r > 1$, to levels $l = 0, \dots, L$, and each level uses a dense square map with cost proportional to n_l^2 . Treat these dimensions as idealized; an implementation must choose integers. The algebraic cost is

$$\sum_{l=0}^L n_l^2 = n_0^2 \frac{1 - r^{-2(L+1)}}{1 - r^{-2}}. \quad (4.2)$$

For fixed $r > 1$ this is bounded as L increases, but still scales as $\mathcal{O}(n_0^2)$. The calculation assumes the representation and the square maps; it does not include learning or communication costs, nor eliminate exponential state-space sampling or approximation complexity in general.

Compression must preserve information needed for the task. Two states with the same position q and opposite velocities v need opposite damping commands $u = -kv$, $k > 0$. A position-only representation maps them to the same value and cannot produce both correct commands. A history-dependent estimator might recover velocity, but then the retained history or estimator state is part of the controller. Hierarchy helps only when its chosen representations and interfaces preserve the distinctions needed downstream.

4.4 Synergies: Reconstruction and Interpretation

For a nonnegative processed EMG vector $\mathbf{e}(t)$, a synchronous spatial model is

$$\mathbf{e}(t) \approx W\mathbf{c}(t), \quad W \in \mathbb{R}_{\geq 0}^{p \times k}, \quad \mathbf{c}(t) \in \mathbb{R}_{\geq 0}^k. \quad (4.3)$$

The residual matters. EMG is a measurement of electrical activity, not a direct identity with a neural control input, activation state, muscle force or joint torque. Time-varying synergy templates are a different model: temporal structure belongs to the templates as well as their recruitment and timing.

d’Avella and Bizzi recorded 13 hindlimb muscles in three frogs during jumping, swimming and walking. Five components gave average reconstruction R^2 values of 0.89 for synchronous synergies and 0.80 for time-varying synergies [11]. Those results concern particular data, preprocessing and model definitions; they do not establish a universal four-to-six-synergy, over-90% rule for human golf. The two representations also differ in parameterization, so component count alone is not a capacity comparison.

Tresch and colleagues compared factorization methods with simulated data and withdrawal EMG from frogs after spinalization. Recovery depended on assumptions about noise and the factors; different criteria could suggest different model orders [43]. Agreement across methods can strengthen a description of data structure, but cannot identify its unique neural generator. For example, WD and $D^{-1}\mathbf{c}$ give the same reconstruction for any positive diagonal D , and component permutations also leave the product unchanged. Further ambiguities can remain. Task restrictions, mechanics and processing choices must be considered alongside a neural-module interpretation.

4.4.1 A Reproducible Factorization Example

Let rows of X index time and columns index channels, so $X \approx CW^T$. The following nonnegative least-squares example uses multiplicative updates [28]. The joint optimization is not convex. The ideal updates’ nonincreasing objective does not guarantee a global optimum, unique factors or exact floating-point monotonicity in this implementation.

Define the *uncentered reconstruction score*

$$S = 1 - \frac{\|X - CW^T\|_F^2}{\|X\|_F^2}.$$

This convention is sometimes called uncentered variance accounted for. It is not centered statistical R^2 ; it is at most one, may be negative, and is undefined for an all-zero dataset.

A high training score alone neither selects a unique rank nor establishes out-of-sample performance.

```

1 import numpy as np
2
3
4 def positive_integer(value: int, name: str) -> None:
5     """Reject zero, fractional values and booleans for count options."""
6     if (isinstance(value, (bool, np.bool_))
7         or not isinstance(value, (int, np.integer))) or value <= 0:
8         raise ValueError(f"{name} must be a positive integer")
9
10
11 def extract_synergies(
12     emg_data: np.ndarray, n_synergies: int,
13     max_iter: int = 500, tol: float = 1e-6
14 ) -> tuple[np.ndarray, np.ndarray, float]:
15     """Return W, C and uncentered score for finite nonnegative data.
16
17     W: channels by rank; C: samples by rank. Reconstruct with C @ W.T.
18     Rows are samples; columns are channels. Zero data are rejected.
19     This teaching example uses one reproducible local random start.
20     """
21     if np.iscomplexobj(emg_data):
22         raise ValueError("data must be real, not complex")
23     data = np.asarray(emg_data, dtype=float)
24     if (data.ndim != 2 or data.size == 0
25         or not np.all(np.isfinite(data)) or np.any(data < 0)):
26         raise ValueError("data must be finite, nonnegative and nonempty 2D")
27     positive_integer(n_synergies, "n_synergies")
28     positive_integer(max_iter, "max_iter")
29     if not np.isfinite(tol) or tol < 0:
30         raise ValueError("tol must be finite and nonnegative")
31     scale = np.max(data)
32     if scale == 0:
33         raise ValueError("relative reconstruction score needs nonzero data")
34     normalized = data / scale
35     rng = np.random.default_rng(42)
36     weights = rng.uniform(0.1, 1.0, (data.shape[1], n_synergies))
37     coefficients = rng.uniform(0.1, 1.0, (data.shape[0], n_synergies))
38     denominator_floor = np.finfo(float).tiny
39     total = np.sum(normalized**2)
40     previous_score = -np.inf
41     for _ in range(max_iter):
42         coefficients *= (normalized @ weights) / np.maximum(
43             coefficients @ (weights.T @ weights), denominator_floor)
44         weights *= (normalized.T @ coefficients) / np.maximum(
45             weights @ (coefficients.T @ coefficients), denominator_floor)
46         residual = normalized - coefficients @ weights.T
47         score = float(1 - np.sum(residual**2) / total)
48         if abs(score - previous_score) < tol:
49             break
50         previous_score = score
51     # Fit normalized data; split its original scale across factors.
52     return weights * np.sqrt(scale), coefficients * np.sqrt(scale), score
53
54
55 rng = np.random.default_rng(7)
56 true_weights = rng.uniform(0.0, 1.0, (8, 3))
57 time = np.linspace(0, 2 * np.pi, 200)
58 true_coefficients = np.maximum(

```

```

59     0, np.sin(time[:, None] + np.arange(3) * 2 * np.pi / 3)
60 )
61 # Arbitrary channels and positive-biased synthetic noise, not walking data.
62 synthetic = (true_coefficients @ true_weights.T
63             + 0.05 * np.abs(rng.normal(size=(200, 8))))
64 reconstruction_scores = [extract_synergies(synthetic, rank)[2]
65                          for rank in range(1, 7)]

```

Listing 4.2: Validated Nonnegative Factorization of Synthetic Channels

The three generating factors are a construction, not an experimental discovery. Channel numbers have no assigned muscle anatomy. Additional fitted components can fit noise; rank inferred from a training-score threshold need not equal the generating count. The Gram-matrix parentheses avoid constructing a channel-by-sample matrix in the weight update; the benefit depends on sample count and rank.

For an empirical study, compare explicitly recorded random starts, preprocessing and model orders. Fit normalization and the basis on training data only; for held-out reconstruction, keep the basis fixed and estimate only permitted coefficients. Distinguish that reconstruction from forecasting future EMG. Check robustness across trials, tasks and subjects. The listing’s one fixed seed is reproducibility for a demonstration, not such a model-selection study.

4.5 From This Hypothesis to a Golf Experiment

A useful swing model connects mechanical state, actuator state and controller state. Joint and club positions alone generally omit velocity, muscle activation, tendon deformation and relevant history. Internal models (Chapter 6) can estimate missing state and predict its evolution. Passive mechanics (Chapter 7) begin responding before a new neural command arrives, while activation and mechanical history change the available force. The coupled-state treatment in Chapter 8 explains why local mechanical response, whole-body stability and task accuracy are separate questions.

Ground-force, hand-motion and club-speed peaks are outputs of that coupled system. Their ordering does not count neural layers, and a compact description of their coordination does not uniquely identify the controller. A controller that uses fewer freshly evaluated stages could reduce delay under a particular implementation; a deeper or recurrent controller might compensate with better estimation or prediction. These alternatives must be tested at the same task, information access and physical constraints.

Key Insight 4.2. The Shallow-Wide Hypothesis

When a correction requires new information under a tight deadline, reducing the critical path can help *if* the resulting representation preserves the information and policy accuracy needed for the task. Test this conditional claim with measured latency, trained controllers, held-out perturbations and matched actuator limits. It is an engineering hypothesis, not a universal layer-count law or proof of a human neural mechanism.

A discriminating experiment varies perturbation predictability, sensing delay, available state variables and controller architecture separately. Report input-to-command and input-to-force latencies, tracking error, delivered club state, contact feasibility and robustness.

Compare parameter count alongside actual compute and communication costs. A benefit under those conditions can support a model choice; translating it to human control requires corresponding physiological measurements and causal tests.

Exercises

1. **Endpoint and Budget.** For the stipulated 50/20/5 ms example, compute the allowable stage count. Repeat with 15 ms available. Explain why neither answer estimates the number of synapses in a golfer.
2. **Count and Measure.** Verify both parameter counts, then measure runtime on a declared machine. Explain why the measured ratio need not be five and why untrained outputs cannot rank control performance.
3. **Lost State.** Construct two equal-position states requiring opposite damping commands. Identify the additional information a compressed controller or its estimator must retain.
4. **Factorization.** Reconstruct the synthetic channels at ranks one through six. Rescale or permute fitted factors without changing their product. Explain what training score, held-out reconstruction and neural identification each establish.
5. **Controlled Comparison.** Design a reaching or golf simulation with identical sensing and actuator constraints. Separate a predictable disturbance from an unannounced one; compare trained policies with measured delays and state explicitly what would remain untested in humans.

Chapter 5

Ideomotor Theory and the Predictive Brain

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Every representation of a movement awakens in some degree the actual movement which is its object.

— William James, 1890

5.1 Historical Ideomotor Theory

The ideomotor principle, articulated by William James in 1890 [22], states that the mere *idea* of a movement tends to produce that movement. Modern reformulations by Greenwald (1970) and Hommel et al. (2001) have transformed this philosophical observation into a testable scientific framework.

Definition 5.1. The Theory of Event Coding (TEC)

The Theory of Event Coding [19] proposes that:

1. Perception and action share a **common representational format** (event codes or feature codes)
2. Actions are planned in terms of their **desired sensory effects**, not in terms of muscle commands
3. The mapping from desired effects to motor commands is learned through experience (bidirectional action-effect associations)

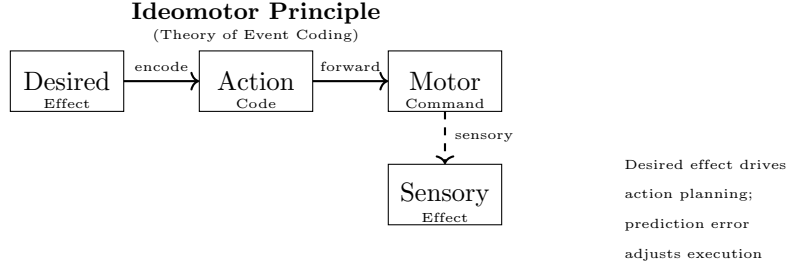


Figure 5.1: The ideomotor principle: actions are planned in terms of their desired sensory effects rather than muscle commands. The forward model predicts the sensory consequence, and prediction error corrects the action.

5.2 The Predictive Processing Framework

Modern neuroscience interprets the brain as a **prediction machine** (Clark, 2013; Hohwy, 2013):

Key Insight 5.1. The Brain as a Prediction Machine

The brain continuously generates predictions about incoming sensory data. Neural processing is dominated by **prediction error minimization**:

$$\varepsilon_k = \mathbf{y}_k - \hat{\mathbf{y}}_k = \mathbf{y}_k - h(\hat{\mathbf{x}}_k), \quad (5.1)$$

where $\hat{\mathbf{y}}_k$ is the predicted observation and $\mathbf{y}_k$ is the actual observation. This error drives updates to the internal model.

In the language of Volume I, Chapter 6: this is exactly the **Kalman filter innovation**. The brain implements Bayesian state estimation.

5.2.1 Active Inference

Karl Friston's active inference framework (Friston, 2010) extends predictive processing to action: instead of only updating beliefs to match observations, the agent acts to make observations match predictions. This formalizes the ideomotor principle in terms of Free Energy minimization:

$$\mathbf{u}^* = \arg \min_{\mathbf{u}} \mathbb{E} \left[\sum_{k=0}^T \|\mathbf{y}_k - \hat{\mathbf{y}}_k\|_{R^{-1}}^2 + \|\mathbf{u}_k\|_Q^2 \right], \quad (5.2)$$

which is formally equivalent to model predictive control (Volume I, Chapter 5).

5.3 Connection to Control Theory

Neuroscience Term	Control Theory Term
Forward model	State transition matrix / predictor
Prediction error	Innovation (Kalman filter)
Efference copy	Feedforward control signal
Active inference	Model predictive control
Prior beliefs	State constraints / initial conditions
Precision weighting	Kalman gain / observation weights
Free energy	Variational cost functional

Exercises

1. **Conceptual.** Explain how the ideomotor principle relates to trajectory-centric control (Volume II). When a golfer imagines impact, what “prediction” are they generating?
2. **Kalman Connection.** Write out the Kalman filter equations and identify which term corresponds to the prediction error in the predictive processing framework.
3. **(Programming.)** Implement an active inference agent that controls a pendulum by minimizing prediction error rather than tracking a reference.

Chapter 6

Internal Models: Forward and Inverse

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

To control is to predict. To predict is to have a model. To have a model is to have learned.

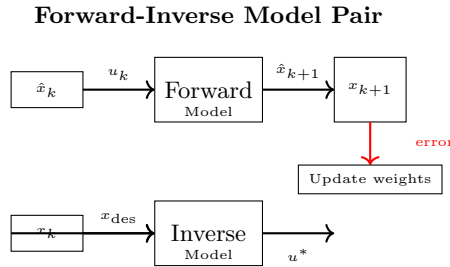


Figure 6.1: Paired forward and inverse models: forward model predicts state evolution from command; inverse model computes command to reach desired state. Learning updates both based on sensory prediction error.

6.1 Forward Models

A forward model predicts the sensory consequences of a motor command:

$$\hat{\mathbf{x}}_{k+1} = f(\hat{\mathbf{x}}_k, \mathbf{u}_k), \quad (6.1)$$

which is exactly the state propagation used in Volume I.

6.1.1 The MOSAIC Architecture

The MOSAIC model (Modular Selection and Identification for Control) [44] proposes that the brain maintains multiple paired forward-inverse models, each tuned to different dynamics or contexts. The claim that a model is acquired *per tool* rather than adjusted in place has direct imaging support: cerebellar activity acquired while learning a novel visuomotor tool persists as a tool-specific representation, separable from the activity driven by the movement itself [21].

```

1 import numpy as np
2 from typing import List, Tuple
3
4 class ForwardInversePair:
5     """A paired forward-inverse model for one context."""
6     def __init__(self, A: np.ndarray, B: np.ndarray):
7         self.A = A # State transition
8         self.B = B # Input matrix
9
10    def forward_predict(self, x: np.ndarray,
11                       u: np.ndarray) -> np.ndarray:
12        """Predict next state."""
13        return self.A @ x + self.B @ u
14
15    def inverse(self, x: np.ndarray,
16              x_desired: np.ndarray) -> np.ndarray:
17        """Compute control to reach desired state."""
18        return np.linalg.lstsq(self.B,
19                              x_desired - self.A @ x,
20                              rcond=None)[0]
21
22
23 class MOSAIC:
24     """Multiple paired forward-inverse model architecture."""
25     def __init__(self, models: List[ForwardInversePair]):
26         self.models = models
27         self.n_models = len(models)
28         self.responsibilities = np.ones(self.n_models) / self.n_models
29
30    def update_responsibilities(self, x: np.ndarray,
31                              u: np.ndarray,
32                              x_next: np.ndarray) -> None:
33        """Update model responsibilities based on prediction errors."""
34        errors = np.zeros(self.n_models)
35        for i, model in enumerate(self.models):
36            prediction = model.forward_predict(x, u)
37            errors[i] = np.sum((x_next - prediction)**2)
38
39        # Softmax weighting (lower error = higher responsibility)
40        log_resp = -0.5 * errors
41        log_resp -= np.max(log_resp) # numerical stability
42        self.responsibilities = np.exp(log_resp)
43        self.responsibilities /= np.sum(self.responsibilities)
44
45    def predict(self, x: np.ndarray,
46              u: np.ndarray) -> np.ndarray:
47        """Weighted prediction across all models."""
48        prediction = np.zeros_like(x)
49        for i, model in enumerate(self.models):
50            prediction += self.responsibilities[i] * \
51                model.forward_predict(x, u)

```

```

52     return prediction
53
54     def control(self, x: np.ndarray,
55                 x_desired: np.ndarray) -> np.ndarray:
56         """Weighted inverse model output."""
57         u = np.zeros(self.models[0].B.shape[1])
58         for i, model in enumerate(self.models):
59             u += self.responsibilities[i] * \
60                 model.inverse(x, x_desired)
61     return u

```

Listing 6.1: MOSAIC-style multiple model architecture.

6.2 Inverse Models

An inverse model computes the motor command required to achieve a desired state:

$$\mathbf{u}_k = f^{-1}(\mathbf{x}_k, \mathbf{x}_{k+1}^{\text{des}}), \quad (6.2)$$

which corresponds to the feedforward control of Volume II. How such a model could be *acquired* rather than assumed is the contribution of feedback-error learning: the output of a crude feedback controller is used as the training signal for the inverse model, so the feedback loop teaches the feedforward path and then falls silent [24]. Note that this is the inverse-model result; the forward-model evidence is separate, and conflating the two is easy because both trace to the same group.

6.3 Smith Predictor for Neural Delays

The sensorimotor system faces finite neural and sensing delays. The Smith predictor architecture uses a forward model to compensate:

$$\hat{\mathbf{x}}_{\text{current}} = \hat{\mathbf{x}}(t - \Delta t) + \int_{t-\Delta t}^t f(\hat{\mathbf{x}}, \mathbf{u}) d\tau, \quad (6.3)$$

where Δt is the sensory delay. The controller operates on the *predicted current state* rather than the delayed sensory measurement.

Exercises

1. **MOSAIC.** Using the provided code, simulate a system that switches between two dynamics (e.g., carrying an empty cup vs. a full cup). Show how the responsibilities shift.
2. **Smith Predictor.** Implement a Smith predictor for a simple pendulum with a user-chosen sensory delay parameter. Compare performance with and without prediction.
3. **(Research.)** Summarize evidence for multiple internal models in human motor control, citing at least three experimental studies.

Chapter 7

Passive Distributed Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The most efficient movement is one where the body's own mechanics do most of the work, and the nervous system merely guides.

— Adapted from Frans Bosch

7.1 The Frans Bosch Framework

Frans Bosch's work on strength training and coordination [6] provides a practitioner-oriented framework for understanding how elite athletes organize movement. His key insights, translated into the language of this series:

Key Insight 7.1. Self-Organization in Complex Movement

Bosch argues that complex movements are not centrally programmed but **self-organize** through the interaction of:

1. **Task constraints:** the goal shapes the solution space
2. **Organismic constraints:** the body's anatomy and current state
3. **Environmental constraints:** gravity, ground, implements

The role of the nervous system is not to specify every muscle activation but to set the *initial conditions* and *constraints* that allow self-organization to produce the desired movement.

In the language of Volume I: the CNS designs the *drift field* $f(\mathbf{x})$ of the system (through postural configuration and muscle tone) and lets the natural dynamics do the work.

Passive Dynamic Walking

(McGeer, 1990)

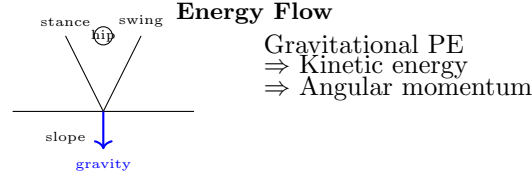


Figure 7.1: Passive dynamic walking: a compass gait on a gentle slope exhibits stable walking cycles with zero control input. Gravity and inertia provide all energy; the walker merely guides the descent.

7.2 Passive Dynamic Walking

The most striking example of passive dynamics is passive dynamic walking (McGeer, 1990): a simple mechanical biped with knees, feet, and hip (no motors) walks down a gentle slope in stable limit cycles with zero active control. This demonstrates that complex coordinated movement can emerge from mechanical design and gravity alone:

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 def passive_walker_dynamics(
5     t: float,
6     state: np.ndarray,
7     M: float = 10.0,
8     m: float = 5.0,
9     l: float = 1.0,
10    g: float = 9.81,
11    slope: float = 0.05
12 ) -> np.ndarray:
13     """Dynamics of a 2-DOF passive walker (compass gait).
14
15     Parameters
16     -----
17     t : float
18         Time.
19     state : np.ndarray
20         [theta_stance, theta_swing, d_theta_stance, d_theta_swing]
21     M : float
22         Hip mass (kg).
23     m : float
24         Leg mass at foot (kg).
25     g : float
26         Gravitational acceleration (m/s^2).
27     slope : float
28         Ground slope (radians).
29
30     Returns
31     -----
32     np.ndarray
33         State derivatives.
34     """
35     th_st, th_sw, dth_st, dth_sw = state

```

```

36
37     # Mass matrix
38     M11 = (M + m) * l**2
39     M12 = -m * l**2 * np.cos(th_st - th_sw)
40     M21 = M12
41     M22 = m * l**2
42
43     # Gravity and Coriolis
44     C1 = -m * l**2 * dth_sw**2 * np.sin(th_st - th_sw) \
45           - (M + m) * g * l * np.sin(th_st - slope)
46     C2 = m * l**2 * dth_st**2 * np.sin(th_st - th_sw) \
47           + m * g * l * np.sin(th_sw - slope)
48
49     # Solve M * ddq = C (NO control input!)
50     det = M11 * M22 - M12 * M21
51     ddth_st = (M22 * C1 - M12 * C2) / det
52     ddth_sw = (M11 * C2 - M21 * C1) / det
53
54     return np.array([dth_st, dth_sw, ddth_st, ddth_sw])
55
56
57 # Simulate one step of passive walking
58 state0 = np.array([0.2, -0.4, -0.5, 1.0])
59 sol = solve_ivp(passive_walker_dynamics, [0, 1.5], state0,
60               max_step=0.001, events=None)
61 print(f"Passive walker: simulated {sol.t[-1]:.2f}s "
62       f"with {len(sol.t)} timesteps")
63 print(f"Final stance angle: {sol.y[0, -1]:.3f} rad")
64 print("(No actuators, no controller --- just gravity and mechanics)")

```

Listing 7.1: Passive dynamic walker simulation.

7.3 Passive Dynamics in the Golf Swing

The golf swing exemplifies passive dynamics exploitation:

1. **Backswing:** energy stored in elastic tissues (muscles, tendons, fascia)
2. **Transition:** elastic recoil initiates the downswing passively
3. **Wrist release:** centrifugal force passively uncocks the wrists (the double pendulum effect from Volume II, Chapter 5)
4. **Follow-through:** passive deceleration through tissue compliance

In the ZTCF family framework of Volume I, Chapter 7: the drift contribution $f(\mathbf{x})$ accounts for $\sim 60\%$ of the clubhead speed. The control contribution $G(\mathbf{x})\mathbf{u}$ provides the remaining $\sim 40\%$.

7.4 Bernstein's Stage 3: Exploiting DOF

The connection to Bernstein's learning framework (Chapter 1) is direct:

- The expert athlete has reached Stage 3 — exploiting degrees of freedom

- They have learned to configure the body so that passive dynamics *automatically* produce the desired motion
- The control problem is simplified: instead of commanding every muscle, set up the initial conditions and let physics do the work

Exercises

1. **Passive Walking.** Using the provided code, find the set of initial conditions that produce a stable limit cycle (periodic gait).
2. **Energy Analysis.** For the passive walker, plot kinetic and potential energy over one stride. Where does the energy come from?
3. **Wrist Release.** Model the wrist release as a passive double pendulum. For what initial angular velocity of the arm does the club reach maximum speed at the bottom?
4. **(Programming.)** Add a small actuator to the passive walker and design a minimal controller that stabilizes the gait on level ground.

Chapter 8

The Interplay of Biology and Dynamics of Nonlinear Systems

8.1 Introduction: Mechanics With a Biological State

A golfer can arrive at nearly the same joint configuration and velocity with different muscle activation, tendon strain and sensory estimates. Those states need not produce the same subsequent motion. Geometry and rigid-body mechanics remain essential, but positions and velocities alone may no longer close the model. The additional state determines how a neural command becomes force, how a disturbance propagates and which responses remain feasible before impact.

This is the useful connection between biology and nonlinear dynamics. Tissue properties, neural feedback, task goals and contact conditions interact through specific equations. Their interaction can support coordination; it can also produce delay, instability, fatigue and estimation error. A biological label does not decide which outcome occurs.

Key Insight 8.1. Three Questions That Must Be Kept Together

1. **Mechanics:** what forces and moments follow from the current geometry, motion, tissue state and contact conditions?
2. **Control:** which excitation or actuator histories can change that state within the remaining time and input limits?
3. **Evidence:** which measurements distinguish the proposed mechanism from another model that produces the same observed movement?

For a swing, the endpoint objective includes the clubhead's delivered position, orientation and velocity, followed by collision dynamics. For a humanoid holding a tool, it also includes grasp feasibility and balance. Neither objective is specified by a single joint torque or a single measure of movement variability.

8.1.1 Bernstein, Constraints and the Scope of the Claim

Having more muscle forces than independent joint moments makes force allocation underdetermined unless additional information or assumptions are supplied. It does not prove that the nervous system cannot coordinate the muscles, nor that it must use a fixed small number of commands. A model's degrees of freedom, actuators, measured EMG channels and task variables are different counts.

Newell grouped constraints into organism, task and environment [34]. Bosch discusses

stable and adaptable movement components in sport [7, 6]. These frameworks motivate hypotheses; formal stability results require a separate model and proof. Bosch’s “fluctuations” can denote flexible task-dependent components; they are not, by definition, expensive errors that must always be suppressed.

An intersection of feasible sets constrains possible states. It does not by itself specify a vector field, an attractor or a controller. Two systems with identical feasible configurations can have different inertia, dissipation and feedback, and therefore different trajectories. Constraints-led practice and predictive or feedback control are compatible descriptions at different levels. Whether a golfer uses a particular internal representation requires evidence beyond a successful dynamical model of the movement.

8.2 Muscle, Tendon and Activation Dynamics

8.2.1 A Consistent Hill-Type Model

Hill’s shortening relation differs from later Hill-type muscle–tendon models [17, 33]. Declare which elements are included, where their lengths are measured and how force is transmitted. In an illustrative massless equilibrium model, a contractile element and passive parallel element share the fiber length l_f . Their force is transmitted through a series tendon, with pennation angle α :

$$F_f = aF_0f_L(l_f/l_0)f_V(\dot{l}_f/v_{\max}) + F_{PE}(l_f), \quad (8.1)$$

$$F_t = F_f \cos \alpha, \quad l_{MT}(q) = l_t + l_f \cos \alpha. \quad (8.2)$$

Here $a \in [0, 1]$ is activation, F_0 the reference active isometric fiber force, l_0 the optimal fiber length, and $v_{\max} > 0$ a shortening-speed scale. The parallel force uses fiber length, not the total muscle–tendon length. Pennation requires its own geometric law if it changes. Fiber and tendon forces are not added as though series elements were parallel force sources.

A simple tension-only tendon law is

$$F_t = k_t[l_t - l_{\text{slack}}]_+, \quad [z]_+ = \max(z, 0), \quad (8.3)$$

with k_t in N/m. The extension is relative to tendon slack length, not muscle length. This piecewise linear law is a teaching approximation; physiological tendon models can include a nonlinear toe region and hysteresis. Series equilibrium and path length together determine fiber velocity or require an implicit solve. Slack tendon and near-zero activation need careful numerical treatment rather than division by a vanishing active-force term.

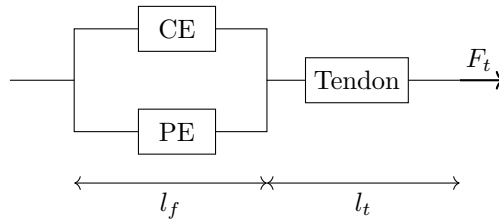


Figure 8.1: Parallel Fiber Elements and a Series Tendon. The Schematic Uses Zero Pennation; a Pennate Model Must Project Fiber Force and Length Consistently.

8.2.2 Length and Velocity Conventions That Can Be Checked

Let $\nu = \dot{l}_f/v_{\max}$, so shortening means $\nu < 0$. One explicit illustrative pair of normalized curves is

$$f_L(\ell) = \exp \left[- \left(\frac{\ell - 1}{w} \right)^2 \right], \quad w > 0, \quad (8.4)$$

$$f_V(\nu) = \begin{cases} \frac{1 + \nu}{1 - \nu/c}, & -1 \leq \nu \leq 0, \\ 1 + (N - 1) \frac{\nu}{d + \nu}, & \nu > 0, \end{cases} \quad c, d > 0, \quad N > 1. \quad (8.5)$$

These equations define the example, not a universal parameter fit or Hill's original eccentric law. They give $f_V(-1) = 0$, $f_V(0) = 1$, and an eccentric limit N . The shortening branch increases with ν ; faster shortening therefore reduces force. Its derivative at zero is $1 + 1/c$. Choosing $d = (N - 1)/(1 + 1/c)$ also matches the eccentric branch's derivative there. Outside the stated domain, choose and validate an extension rather than letting negative active tension silently enter a simulation.

For $c = 0.25$, $N = 1.5$ and $d = 0.1$, $f_V(-0.5) = 1/6$ and $f_V(0.1) = 1.25$. At $a = 0.5$, $F_0 = 1000$ N and optimal length, active fiber force is respectively 83.33 N and 625 N. These are chosen model numbers, not measured human values. Passive force and pennation still affect tendon force. Total force at a long fiber length can exceed active isometric force because the passive element contributes; the active force-length peak alone does not bound the total tension.

8.2.3 Zero Excitation Retains the Activation History

Excitation u and activation a are different variables. For a constant activation time scale $\tau_a > 0$, a simple example is

$$\dot{a} = (u - a)/\tau_a, \quad 0 \leq u \leq 1. \quad (8.6)$$

If u is set to zero at t_0 , then $a(t) = a(t_0) \exp[-(t - t_0)/\tau_a]$. Active force generally persists while activation decays. Real activation/deactivation dynamics can require different time scales and nonlinearities. With activation, fiber length and other internal states z , a closed model uses $x = (q, v, a, z)$, not only $x = (q, v)$.

For muscle path lengths $l_{\text{MT}}(q)$, define $J_l = \partial l_{\text{MT}}/\partial q$. Positive tendon tension pulls toward shorter path length, giving generalized muscle load $-J_l^T F_t$:

$$M(q)\dot{v} + h(q, v) = -J_l(q)^T F_t(x) + Q_{\text{ext}} + A(q)^T \lambda, \quad (8.7)$$

$$\dot{q} = v, \quad \dot{z} = \psi(x), \quad Av = 0. \quad (8.8)$$

Here generalized coordinates are chosen so $\dot{q} = v$; other representations need their kinematic map. h includes the declared gravity and velocity terms, and λ enforces the current independent ideal constraints. Ground friction, grasp limits and changing contact modes need additional conditions.

With constant τ_a and no other direct excitation dependence, the augmented system is control-affine: $\dot{x} = f(x) + G(x)u$. Excitation enters the activation rows; its influence on acceleration develops through the state. The drift f at $u = 0$ includes existing activation, tendon strain and constraint response. Holding a nonzero baseline command instead defines

another vector field. It is legitimate to study that field, but it must not be called fully passive or free of ongoing energetic cost.

8.2.4 Preflex Response and Tendon Energy

An intrinsic mechanical response can precede a newly evoked neural response. At a fixed activation and around a specified state, a local approximation is

$$\delta F_f \simeq K_f \delta l_f + D_f \delta \dot{l}_f. \quad (8.9)$$

The coefficients depend on operating point and history, not just activation. This approximation is often associated with the preflex concept [29]. It introduces no new neural-loop delay; that does not imply instantaneous global force transmission, an inactive nervous system, or guaranteed stability. The sign and size of incremental stiffness, tendon compliance, limb geometry and contact determine the resulting motion.

For example, a damped inverted pendulum near upright obeys

$$I\ddot{\theta} + D\dot{\theta} + (K - mgh)\theta = 0. \quad (8.10)$$

With $I > 0$ and $D > 0$, this linear model is asymptotically stable only when $K > mgh$. Positive tissue stiffness is insufficient if it is smaller than the destabilizing gravitational term. Direct human standing measurements by Loram and Lakie found intrinsic ankle stiffness insufficient for stability in their protocol [30]. This is a useful counterexample to a general claim that baseline muscle tone automatically stabilizes posture. Spinal circuits are part of the CNS; anesthesia does not imply elimination of every reflex. Neither claim is needed to explain a mechanical preflex.

For the elastic law above,

$$E_t = \frac{1}{2}k_t[l_t - l_{\text{slack}}]_+^2, \quad \dot{E}_t = F_t\dot{l}_t \quad (8.11)$$

away from the switching point. A chosen $k_t = 20,000$ N/m and 0.02 m extension store 4 J at 400 N. Releasing 4 J in 0.02 s supplies 200 W of average recoil power in this ideal example. The energy had to be loaded earlier; recoil changes the timing of power without creating energy. Fiber work, skeletal work, tendon storage and losses must close one balance. There is no universal 50% tendon contribution to jumping or striking, and an elastic element is not generally equivalent to rescaling time in a rigid-body solution.

For a golf swing, a catapult hypothesis therefore needs a measured or bounded strain history and release timing. A compliant robot actuator needs the same accounting. More compliance can improve a specified objective or impair force bandwidth, precision and contact control; the answer depends on the task.

8.3 Attractors and Coordination Transitions

8.3.1 What Stability Does and Does Not Establish

For a declared autonomous closed-loop system, we use an asymptotically stable attractor to mean a compact invariant set that is Lyapunov stable and attracts some neighborhood. Lyapunov stability means sufficiently small disturbances remain small; attraction additionally concerns convergence to the set. Definitions of more general attractors differ, so minimality and a single negative exponent should not be substituted for these conditions.

Attraction to a periodic orbit is distinct from convergence to one time-stamped trajectory on that orbit. A stable autonomous limit cycle has a neutral phase direction, with a zero Lyapunov exponent and a unit Floquet multiplier; its transverse multipliers can lie inside the unit circle. Nearby motions can retain a phase offset while approaching the same orbit. A chaotic attractor can attract trajectories to its set while separating nearby trajectories within that set. Finite-time growth, asymptotic exponents and basin size answer different questions.

These distinctions matter in golf. A repeatable swing is a finite-duration movement ending in contact, not automatically a limit cycle or equilibrium. An appropriate analysis can use a trajectory tube, terminal error, input limits and impact-event sensitivity. A phase error that is harmless for convergence to a periodic gait can be consequential when club and ball must meet at one time.

An attractor can be generated by passive mechanics, constant powered actuation, neural feedback or a robot controller. Its existence does not identify which mechanism supplies stability, how much power is consumed or whether a new task can be accommodated. A passive downhill walker draws energy from gravity; an actively stabilized posture can consume energy without moving. A finite feedforward maneuver need not be an attractor to meet an error tolerance.

Example 8.1. A Stable Orbit With a Neutral Phase

On an annulus around $r = 1$, consider $\dot{r} = 1 - r$, $\dot{\theta} = \omega > 0$. Radial deviations decay as e^{-t} , but two solutions' angular difference remains constant. The orbit $r = 1$ attracts nearby states although their time-matched Euclidean separation need not tend to zero. Transverse recovery and phase recovery must therefore be measured separately.

8.3.2 A Cost Crossing Is Not a Dynamical Bifurcation

A dynamical bifurcation changes the qualitative behavior of a vector field or return map as a parameter varies. Two metabolic cost curves crossing does not show that either gait loses dynamical stability. Even in the simple chosen costs $C_1(v) = 1 + v^2$ and $C_2(v) = 2 + v^2/2$, the crossing at $v = \sqrt{2}$ says only which algebraic cost is smaller. It supplies no equations for a gait. An observed walk–run transition can be investigated using energetic, mechanical, perceptual and stability hypotheses; matching one transition speed does not distinguish them. These teaching curves are dimensionless examples, not fitted human metabolic laws.

Likewise, a coordination variable and an experimentally adjusted “control parameter” are not necessarily a plant state and actuator input in the control-theory sense. The relationship must be derived or fitted. Frequency may alter several physiological and mechanical processes at once.

8.3.3 The Haken–Kelso–Bunz Model With Its Correct Transition

In the classic bimanual experiment, increasing cycling frequency can cause the initial anti-phase mode to switch to in-phase coordination, using the authors' homologous-muscle phase convention [15]. Symmetric movements toward and away from the body's midline can be in-phase even though the hands' world-coordinate velocities point in opposite directions. Define the measured phase before using “same direction” as a substitute.

The standard reduced deterministic phase equation is first order:

$$\dot{\phi} = -a_H \sin \phi - 2b_H \sin(2\phi) = -V'(\phi), \quad (8.12)$$

$$V(\phi) = -a_H \cos \phi - b_H \cos(2\phi), \quad a_H, b_H > 0. \quad (8.13)$$

The potential is a reduced dynamical construction, not metabolic or mechanical energy. For fixed parameters, $\dot{V} = -[V'(\phi)]^2 \leq 0$. Linearizing at the two coordination modes gives

$$\lambda_0 = -a_H - 4b_H < 0, \quad \lambda_\pi = a_H - 4b_H. \quad (8.14)$$

Thus in-phase $\phi = 0$ remains locally stable, whereas anti-phase $\phi = \pi$ is stable only for $b_H/a_H > 1/4$. If the ratio decreases through $1/4$ as cycling frequency increases, anti-phase loses stability. A chosen frequency law is required before assigning a transition frequency in Hz. A forced second-order phase oscillator is a different model requiring its own derivation.

For $a_H = 1 \text{ s}^{-1}$, changing b_H from 0.5 to 0.2 s^{-1} changes λ_π from -1 to $+0.2 \text{ s}^{-1}$ while λ_0 remains negative. Exactly $\phi = \pi$ is an equilibrium even after losing stability: a numerical experiment must include a small perturbation or a declared noise process to observe departure. Near the transition, slow recovery and noise-induced switching can complicate identification of the deterministic threshold.

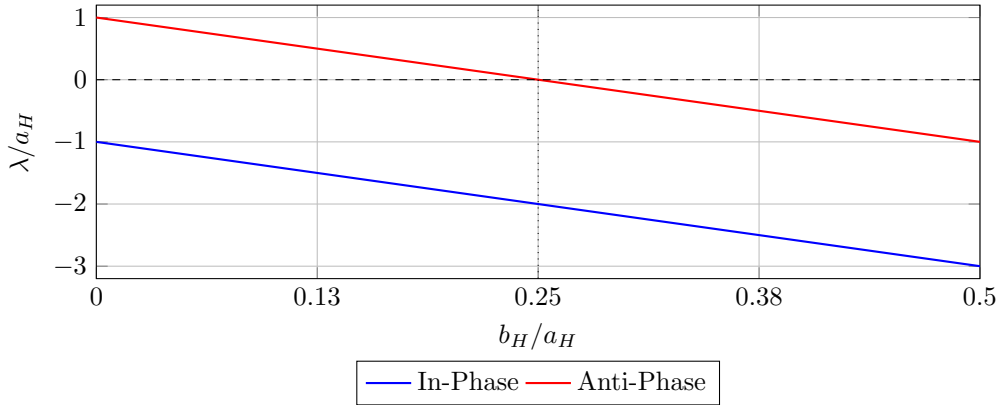


Figure 8.2: Local Phase Stability in the HKB Reduction. Negative Rates Indicate Recovery; Anti-Phase Loses Stability as the Ratio Falls Below One Quarter.

The reduced model explains an experimental coordination transition without an explicit discrete mode-selection command in its equations. It does not show that neurons are absent, that the brain never predicts trajectories or that golf timing follows the same phase law. In a swing, a proposed collective variable needs a measurement definition, fitted dynamics and tests under perturbations and altered tempo. A recognizable pattern is a starting point for that work, not its causal conclusion.

8.4 Synergies, Task Geometry and Control Authority

8.4.1 What a Low-Dimensional Fit Establishes

A spatial muscle-synergy model approximates an EMG or activation matrix $U \in \mathbb{R}^{p \times N}$ by WC , with $W \in \mathbb{R}^{p \times k}$ and $C \in \mathbb{R}^{k \times N}$. Nonnegative factorization usually imposes $W, C \geq 0$.

Specify preprocessing, normalization, muscles, tasks and rank selection; EMG is not a direct measurement of muscle force.

$$\text{VAF}_{\text{uncentered}} = 100 \left(1 - \frac{\|U - WC\|_F^2}{\|U\|_F^2} \right) \%, \quad \|U\|_F > 0. \quad (8.15)$$

The percentage multiplies the entire bracket. A centered variance definition uses a different denominator and must be named. High reconstruction quality is a property of the dataset and fit, not proof of a neural module or a universal number of synergies.

Kutch and Valero-Cuevas supplied counterexamples in which biomechanics produces low-dimensional activation patterns without fixed neural grouping [26]. Such alternatives do not establish that neural synergies never exist. They show why a causal claim needs task variation, perturbation, held-out prediction and comparison with independently controlled muscles. A useful robot policy may intentionally restrict its inputs to synergies even when the corresponding human mechanism remains unresolved.

8.4.2 A Task Level Set and Its Tangent Space

For a smooth task map $y = h(q)$ and target y_* , the exact task-preserving set is

$$\mathcal{M}_{y_*} = \{q : h(q) = y_*\}. \quad (8.16)$$

At a regular point $\bar{q}$ with rank- r Jacobian $J = Dh(\bar{q})$, its tangent space is $\ker J$ and has dimension $n-r$. The local uncontrolled-manifold analysis uses this linear approximation [36]. A finite step in a tangent direction generally leaves the curved level set:

$$h(\bar{q} + \delta q) - h(\bar{q}) = J\delta q + \frac{1}{2}D^2h(\bar{q})[\delta q, \delta q] + O(\|\delta q\|^3). \quad (8.17)$$

For $h(q) = q_1^2 + q_2^2$, $\bar{q} = (1, 0)$ and $\delta q = (0, 0.1)$, $J\delta q = 0$ but the task value increases from 1 to 1.01. A singular point requires a separate analysis; a changing Jacobian rank invalidates one fixed dimension formula across the whole region.

Using a declared Euclidean metric, let $P_N = I - J^\dagger J$ and $P_R = J^\dagger J$. For joint-deviation covariance Σ , dimension-normalized variances are

$$V_{\text{UCM}} = \frac{\text{tr}(P_N \Sigma)}{n - r}, \quad V_{\text{ORT}} = \frac{\text{tr}(P_R \Sigma)}{r}, \quad 0 < r < n. \quad (8.18)$$

Coordinates mixing lengths and angles require meaningful scaling or another metric. For isotropic noise $\Sigma = \sigma^2 I$, both normalized variances equal σ^2 ; redundancy alone does not give a ratio greater than one. Repeated trials also need alignment, a task definition and a treatment of measurement noise. More variance in the tangent directions supports a particular task-stabilization hypothesis; it does not establish that these directions are neurologically uncontrolled or irrelevant to every goal.

In golf, a null direction for hand position may change face orientation, shaft strain, grip loading or a later collision. The chosen output should therefore reflect the actual question. Kinematic task-preserving variations and muscle activation synergies live in different spaces; relating them requires muscle geometry and dynamics rather than identifying the two subspaces by name.

8.4.3 Instantaneous Rank Is Not Controllability

If an input policy imposes $u = Wc$ in $\dot{x} = f(x) + G(x)u$, the instantaneous input map is GW , with rank at most k . Nonnegative and bounded amplitudes further limit the feasible directions to a cone or bounded set. This does not mean that the other state directions are uncontrollable over time.

For the double integrator $\dot{q} = v$, $\dot{v} = u$, the instantaneous input matrix is $B = (0, 1)^T$, but

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad [B, AB] = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (8.19)$$

has rank two. Drift transports the input's effect into position. Along a general nominal motion, use the linearized pair $A(t), B(t)$ and, where appropriate, its finite-horizon Gramian. Nonlinear accessibility, finite-time reachability and controllability under bounds are distinct additional questions.

Nor does a direction invisible in the current output remain unobservable. For the same double integrator with output $y = q$, velocity is initially in $\ker C$, $C = (1, 0)$, yet $[C^T, (CA)^T]^T$ has rank two. Position measured over time reveals velocity in the ideal model. There is no theorem that discarded synergy directions must coincide with an unobservable UCM. The practical test is whether the restricted policy can meet the chosen task and disturbance tolerances within its available time and force limits.

8.5 Impedance and Co-Contraction

8.5.1 Force, Displacement and Velocity Ports

For a local scalar perturbation model around a fixed operating point,

$$M\delta\ddot{x} + D\delta\dot{x} + K\delta x = \delta F, \quad M, D, K > 0, \quad (8.20)$$

the zero-initial-condition Laplace relations are

$$S(s) = \frac{\delta F(s)}{\delta X(s)} = Ms^2 + Ds + K, \quad (8.21)$$

$$Z(s) = \frac{\delta F(s)}{\delta V(s)} = Ms + D + K/s, \quad (8.22)$$

$$H(s) = \frac{\delta X(s)}{\delta F(s)} = \frac{1}{Ms^2 + Ds + K}. \quad (8.23)$$

S is dynamic stiffness (N/m); Z is mechanical impedance (Ns/m); H is displacement compliance. Some robotics literature uses “impedance control” for enforcing the complete mass–damping–stiffness relation. Naming that strategy does not remove the distinction between the transfer functions and their units. Multi-axis impedance needs matrices, a frame and a power-consistent port.

The natural frequency is $\omega_n = \sqrt{K/M}$ and damping ratio $\zeta = D/(2\sqrt{MK})$. Increasing K at fixed D raises ω_n but lowers ζ . For the chosen $M = 1$ kg, $D = 10$ Ns/m and $K = 100$ N/m, $\omega_n = 10$ rad/s and $\zeta = 0.5$. Increasing K to 400 N/m doubles the natural frequency and halves the damping ratio. A faster natural time scale does not guarantee less overshoot, smaller contact forces or better performance.

8.5.2 How Muscle Forces Contribute to Joint Stiffness

At fixed activation and internal-state assumptions, take $\tau(q) = -\sum_i F_i(l_i(q))\nabla l_i(q)$. Linearized restoring stiffness is

$$K_q = -\frac{\partial \tau}{\partial q} = \sum_i [k_i \nabla l_i \nabla l_i^T + F_i \nabla^2 l_i], \quad k_i = \frac{\partial F_i}{\partial l_i}. \quad (8.24)$$

The first term is material stiffness transmitted through moment arms; the second is geometric stiffness from loaded, changing geometry. Muscle–tendon series compliance changes the effective k_i . Coupled muscle dynamics, reflexes and active feedback require a frequency-dependent extension.

In a chosen one-joint model with constant opposing moment arms of 0.03 m, equal forces of 100 N produce zero net torque. If both effective longitudinal stiffnesses are 10,000 N/m, the joint stiffness is $2(0.03)^2(10,000) = 18$ N m/rad. The antagonists carry real load even though their net torque cancels. Equal activation need not produce equal opposing torques when muscle sizes, lengths or moment arms differ.

Hogan’s analysis and forearm-posture experiment support a role for antagonist coactivation in impedance modulation [18]. They do not establish a universal stiffness proportional to target uncertainty. Target uncertainty, external force disturbances, observation noise and activation-dependent noise enter different parts of a control problem. Increasing stiffness can resist a force disturbance while increasing the force caused by a position error against a stiff environment. The disturbance and the scored outcome must be named.

8.5.3 An Explicit Optimization Example

For a quasistatic scalar model $e = w/K$ with zero-mean force disturbance of variance σ_F^2 , suppose a designer chooses the surrogate

$$J(K) = \frac{\sigma_F^2}{K^2} + \rho K^2, \quad 0 < K_{\min} \leq K \leq K_{\max}, \quad \rho > 0. \quad (8.25)$$

Weights carry the units needed to compare the terms. The first term is expected squared displacement; the second is an assumed stiffness penalty, not measured metabolic energy. Before applying bounds, differentiation gives

$$K_0 = (\sigma_F^2/\rho)^{1/4}, \quad K^* = \text{clip}(K_0, K_{\min}, K_{\max}). \quad (8.26)$$

For $\sigma_F = 10$ N, $\rho = 10^{-6}$ m⁴/N² and bounds 20–150 N/m, $K^* = 100$ N/m. Four times the force standard deviation would make the unconstrained optimum 200 N/m and the bounded optimum 150 N/m. Maximum feasible stiffness can therefore be optimal for a stated task. Changing the penalty, disturbance spectrum or dynamics changes the result; there is no general linear “signal-to-noise” stiffness law.

For the stable compliance H above, $\|H\|_\infty = \sup_\omega |H(i\omega)|$ measures a frequency-domain gain and the induced L_2 gain under the usual zero-initial-condition assumptions. It does not guarantee bounded output for arbitrary unlimited disturbances or robustness to unspecified model uncertainty. For this scalar system,

$$\|H\|_\infty = \begin{cases} 1/K, & \zeta \geq 1/\sqrt{2}, \\ 1/[2\zeta K \sqrt{1-\zeta^2}], & 0 < \zeta < 1/\sqrt{2}. \end{cases} \quad (8.27)$$

The dynamic-stiffness polynomial S instead grows without bound at high frequency; it is not the stable proper disturbance-to-displacement transfer whose finite norm was intended. In a real controller, sensing, filters, delay, contact and actuator dynamics determine the relevant closed-loop transfer.

8.5.4 Mechanical Work and Metabolic Cost

The local passive model stores $E = \frac{1}{2}M\delta\dot{x}^2 + \frac{1}{2}K\delta x^2$ and, for constant M, K , obeys $\dot{E} = \delta F\delta\dot{x} - D\delta\dot{x}^2$. If stiffness is varied, an additional $\frac{1}{2}\dot{K}\delta x^2$ term enters the energy balance. An actively changed spring parameter can supply energy; passivity cannot be inferred from positive instantaneous K alone.

Physiological metabolic cost also depends on activation, contraction history, force and velocity, tissue properties and the selected metabolic model. It is not generally proportional to $\int K dt$, net joint work, or squared inverse-dynamics torque. A golfer can have low net wrist torque while opposing muscles are loaded. A robot can hold a load with zero joint work while its motor dissipates electrical energy. Report the measured or modeled cost actually used; mechanical cancellation does not make physiological effort disappear.

8.6 Pre-Stress and Stability of Interacting Subsystems

8.6.1 What a Tensegrity Analogy Can Contribute

Pre-stress means an internal load state present before an additional perturbation. A self-stress in a free structural model balances internally without external nodal load. More generally, a loaded structure is linearized around an equilibrium that includes its external loads. Neither definition says that “internal forces exceed external forces”; these are different force collections, not one scalar comparison.

For a declared elastic potential $U(q)$, the constrained tangent stiffness comes from the appropriate second variation of total potential at equilibrium. Material and geometric contributions both matter, as in the preceding muscle-path formula. Preloading can stiffen some directions and soften others, including buckling. Equilibrium force balance alone does not prove stability.

Tensegrity models can help formulate questions about distributed load paths. Anatomical bones also experience bending, shear and joint-contact forces, while muscles, tendons and fascia have distributed geometry and constitutive behavior. A cell-scale or ideal strut-cable model is not by itself validation of a whole-body architecture. Rigid-body multibody models can already include muscle forces, compliance and loaded constraints; these modeling approaches need not be treated as mutually exclusive.

For golf, useful questions concern the force path from ground through legs, trunk, arms and grip, and the resulting tangent response under a defined perturbation. Muscle tone supplies force and may change that response; it is not separate from force generation. A proposed fascia-mediated mechanism must predict an observable effect beyond what the competing joint/contact model already explains. A humanoid comparison likewise needs a specified structure, actuation and contact model, rather than an anatomical analogy alone.

8.6.2 Why Stable Parts Can Form an Unstable Whole

Organizing analysis into tissue, intermuscular and task levels is useful. Their time scales overlap, however, and their couplings must remain in the equations. A spinal feedback loop is a CNS process, and faster mechanics does not eliminate sensing, prediction or adaptation. Separate stability does not guarantee that the sum of the levels' Lyapunov functions is a composite certificate: coupling terms must also be controlled.

Example 8.2. An Unstable Interconnection of Stable Scalar Parts

Consider

$$\dot{z}_1 = -z_1 + 2z_2, \quad \dot{z}_2 = -z_2 + 2z_1. \quad (8.28)$$

Each isolated equation decays when the other state is fixed at zero. The coupled matrix has eigenvalues 1 and -3 . For $V = (z_1^2 + z_2^2)/2$, $\dot{V} = -z_1^2 - z_2^2 + 4z_1z_2$ is positive at $z_1 = z_2 \neq 0$. Separate stability has not controlled the feedback interaction.

Theorem 8.1 (A Sufficient Composite Decay Condition). *Suppose positive-definite local functions V_i for a common equilibrium satisfy, on a specified neighborhood,*

$$\dot{V}_i \leq -\alpha_i \|z_i\|^2 + \sum_{j \neq i} \gamma_{ij} \|z_i\| \|z_j\|, \quad \alpha_i > 0, \quad \gamma_{ij} \geq 0.$$

If weights $w_i > 0$ make the symmetric matrix with entries

$$S_{ii} = w_i \alpha_i, \quad S_{ij} = -\frac{1}{2}(w_i \gamma_{ij} + w_j \gamma_{ji})$$

positive definite, then $V = \sum_i w_i V_i$ has strictly negative derivative away from the equilibrium in that neighborhood. With the usual regularity and local positive-definiteness bounds, it proves local asymptotic stability.

Proof. Set $r_i = \|z_i\|$. Summing the derivative bounds gives $\dot{V} \leq -r^T S r \leq -\lambda_{\min}(S) \|r\|^2$. Small enough sublevel sets remain inside the neighborhood; the Lyapunov criterion then applies. A global conclusion would require global hypotheses. $\square$

For three scalar levels with diagonal decay one and symmetric neighbor coupling 0.2, the decay matrix has smallest eigenvalue $1 - 0.2\sqrt{2} \simeq 0.7172 > 0$. This chosen example passes the condition. It is not an estimate of a nervous system's coupling strength. Other systems may need input-to-state stability, small-gain, passivity or singular-perturbation analysis. For contact switches, also check reset-induced changes in the certificate; continuous decay between events is not a complete hybrid stability proof.

The resulting research claim is specific: suitable coupling and feedback can make distributed stabilization work. A hierarchy is an analysis strategy, not automatic protection from dimensionality, delay or instability.

8.7 Building and Testing a Coupled Movement Model

8.7.1 Choose the Model for the Question

Inverse dynamics recovers the net generalized loads consistent with specified kinematics, inertia and external loads. It does not inherently assume that muscle activation is instantaneous or that tendons are rigid: those assumptions belong to a chosen actuator model.

Recovering a joint moment does not identify the individual muscle forces producing it. Ballistic movements can be particularly sensitive to activation history, elasticity and contact timing; speed alone does not justify ignoring those states.

A practical model comparison starts with a rigid-body net-load model, then adds only the states needed to answer the question. A muscle-driven model can add activation, fiber and tendon dynamics; a humanoid actuator model can add motor current, transmission compliance and control delay. Flexible shafts and grips require corresponding deformation states. Both versions should retain the same task, initial-condition specification and external-load convention.

Check equilibrium, power balance, force bounds, constraint residuals and numerical convergence before interpreting differences. A more elaborate model can fit more observations while being less identifiable. Adding springs or feedback does not guarantee useful attractors, favorable energy transfer or better predictions. Compare the simplest competing models against held-out data.

8.7.2 Identification Requires More Than a Motion Fit

Let measurements satisfy $y_k = \mathcal{H}(x(t_k), \theta) + \epsilon_k$, with parameters θ . The local sensitivity matrix

$$\mathcal{S}_{kj} = \frac{\partial y_k}{\partial \theta_j} \quad (8.29)$$

reveals combinations distinguishable in the selected experiment. When columns are nearly dependent, different parameters can compensate for one another. Measurement covariance, parameter scaling and model discrepancy all affect the interpretation. A successful optimizer does not establish physiological uniqueness or remove uncertainty.

Joint motion and ground reactions constrain net mechanics. EMG adds timing and activation-related evidence with its own calibration limitations. Ultrasound or other tissue measurements can constrain fiber or tendon behavior when the measurement is appropriate. Perturbation experiments can separate stiffness, damping and delayed feedback more effectively than repeated unperturbed motion. No fixed parameter count guarantees identifiability, and imaging is not uniformly invasive. Specify what each measurement adds and what remains inferred.

For learning and fatigue, parameters or policies may change across trials. Separate within-trial state estimation from across-trial adaptation. A model that refits each swing independently can conceal a failure to predict how a changed grip, tempo, load or support condition changes the response.

8.7.3 A Golf and Humanoid Experiment Ladder

1. **Define the delivered state.** Record the clubhead or tool frame, reference point, contact event and task tolerance. State whether the objective is face control, speed, balance, disturbance recovery or a combination. Joint angles alone do not specify all of these outcomes.
2. **Define comparable initial conditions.** Match or estimate posture, velocity, activation and elastic state. Distinguish a new command from a changed starting state. A zero-excitation continuation preserves the existing activation and strain history; it is not an instruction to make the person instantaneously relaxed.

3. **Apply a declared disturbance.** Vary tool inertia, support, grip loading or a measured external perturbation within an appropriate protocol. Specify the time and direction. Disturbances applied before and near impact probe different available correction horizons.
4. **Measure both task and internal response.** Compare delivered state, phase error, hand/ground loads and relevant activation or strain evidence. A successful endpoint with larger opposing forces may trade robustness for effort. Lower joint torque alone cannot decide whether that trade occurred.
5. **Compare mechanisms.** Test a net-load model, an augmented tissue or actuator model, and alternative feedback policies using the same scored data. Report prediction intervals and failures, including cases in which added compliance or reduced input dimension worsens the task.

These comparisons expose how the components are intertwined. Posture changes moment arms and inertia; activation and strain change force and local response; contact changes feasible reactions; those changes alter the useful feedback directions and remaining time. Earlier work can become later kinetic or elastic energy, while a change that improves speed may impair force control or loading. The joint explanation is a sequence of measurable consequences, not a claim that mechanics replaces neural control.

8.7.4 What Would Count Against the Proposed Explanation?

A reflex-based explanation weakens if the estimated intrinsic response is too small or has the wrong sign to explain early recovery. A tendon-recoil explanation weakens if the bounded recoverable strain energy is insufficient or is released at the wrong time. A fixed-synergy explanation weakens if it fails to predict withheld task directions or perturbations that an independently actuated model predicts. An attractor explanation weakens if the supposed recovery disappears after phase alignment or depends on unmodeled powered feedback. A hierarchical stability claim fails if cross-coupling or contact resets defeat its proposed certificate.

These are model-specific tests, not reasons to reject biological coordination as a whole. They make the useful positive argument stronger: a successful model explains both why a pattern works in its operating region and why it stops working when the relevant conditions change.

8.8 Connection to the Series

Geometry supplies configurations, tangent spaces and moment arms. Multibody dynamics translates applied loads into motion. Muscle and actuator models describe how command history becomes load and impedance. Feedback, prediction and learning shape behavior within those physical limits. Contact and impact determine how the delivered state becomes task outcome. Each layer adds an assumption and a corresponding opportunity for validation.

The neighboring treatments of passive control, central pattern generation, motor learning and robotics should be read with this separation in mind. Distributed mechanics and neural computation can cooperate; a low-dimensional description can be useful without uniquely

identifying a neural implementation. The strongest golf-swing argument specifies the state, intervention, constraints, time horizon and observable consequence. The same discipline supports a humanoid-tool model while preserving the differences in its actuators and sensing.

Exercises

1. **Force–Velocity and Series Equilibrium.** Verify the values and one-sided derivatives of the stated f_V at zero. Use the declared force–length and tension-only tendon laws with zero pennation to solve a prescribed length-history example. Increasing fiber length is lengthening, not shortening. Report how tendon compliance and activation time scale separately affect the result; do not prescribe inconsistent fiber and tendon lengths independently.
2. **Stored Energy and Zero Excitation.** Reproduce the 4 J tendon example and its ideal average recoil power. Then use $a(t_0) = 0.8$ and a chosen $\tau_a = 0.04$ s to calculate activation 0.04 s after zeroing excitation. Explain why the resulting $0.8/e \simeq 0.2943$ need not produce zero active force. Close the energy balance when losses are added.
3. **Orbit and Trajectory Stability.** Simulate the radial/phase example from two nearby initial phases and radii. Plot distance to the orbit and time-matched distance between trajectories. Explain which measure is appropriate for a periodic rhythm and which additional timing information a collision task requires.
4. **HKB Transition.** Use the first-order phase equation with $a_H = 1$ and decrease b_H through 0.25. Start near, rather than exactly at, π . Compare the observed recovery rate with $a_H - 4b_H$ and test sensitivity to perturbation size and parameter-ramp speed. A transition frequency cannot be reported until a frequency-to-parameter relationship has been specified.
5. **Uncontrolled-Manifold Null Model.** At a regular configuration of a three-link planar arm, compute a rank-two position Jacobian and the dimension-normalized covariance projections. First use isotropic joint noise: the population ratio is one, not automatically greater than one. Then impose a larger covariance in the tangent direction. Compare linearized task variation with the actual nonlinear forward kinematics as perturbation size increases. Repeat with orientation included in the task.
6. **Impedance and Coupling Trade-Offs.** Reproduce the bounded stiffness optimum, then compare disturbance compliance and contact force under a prescribed position error. Verify the two-level unstable interconnection and the three-level sufficient decay condition. Finally introduce a delay or reset and determine which earlier certificate must be reconsidered. A favorable result from one objective is not a universal ranking of stiff and compliant strategies.

Chapter 9

Central Pattern Generators

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Rhythm is the most ancient form of motor control.

9.1 CPGs in Biological Motor Control

Central pattern generators are neural circuits that produce rhythmic motor output without rhythmic sensory input or supraspinal commands. Evidence for CPGs:

- Isolated lamprey spinal cord generates swimming patterns [8]
- Deafferented cats produce coordinated stepping on treadmills [14]
- Songbird pre-motor nuclei show rhythmic activity in vitro [40]

CPGs underlie locomotion, respiration, chewing, and other rhythmic behaviors.

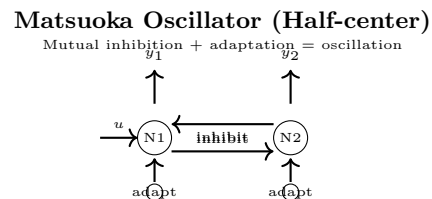


Figure 9.1: Matsuoka oscillator CPG circuit: mutual inhibition between neurons with adaptation variables creates an alternating rhythm. Tonic input frequency-modulates the oscillation without losing periodicity.

9.2 The Matsuoka Oscillator

The most widely used CPG model in robotics is the Matsuoka oscillator [32]:

$$\tau_1 \dot{x}_i = -x_i - \beta v_i - \sum_{j \neq i} w_{ij} y_j + u_i + s_i, \quad (9.1)$$

$$\tau_2 \dot{v}_i = -v_i + y_i, \quad (9.2)$$

$$y_i = \max(0, x_i), \quad (9.3)$$

where x_i is the membrane potential, v_i is the adaptation variable, y_i is the output, w_{ij} are inhibitory coupling weights, u_i is the tonic input, and s_i is the sensory feedback.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 class MatsuokaOscillator:
5     """Two-neuron Matsuoka oscillator for CPG-based control."""
6
7     def __init__(
8         self,
9         tau1: float = 0.1,
10        tau2: float = 0.3,
11        beta: float = 2.5,
12        w_mutual: float = 2.0,
13        tonic_input: float = 1.0
14    ):
15        self.tau1 = tau1
16        self.tau2 = tau2
17        self.beta = beta
18        self.w = w_mutual
19        self.u = tonic_input
20
21    def dynamics(self, t: float, state: np.ndarray) -> np.ndarray:
22        x1, v1, x2, v2 = state
23        y1 = max(0.0, x1)
24        y2 = max(0.0, x2)
25
26        dx1 = (-x1 - self.beta * v1 - self.w * y2 + self.u) / self.tau1
27        dv1 = (-v1 + y1) / self.tau2
28        dx2 = (-x2 - self.beta * v2 - self.w * y1 + self.u) / self.tau1
29        dv2 = (-v2 + y2) / self.tau2
30
31        return np.array([dx1, dv1, dx2, dv2])
32
33    def simulate(self, t_end: float = 5.0,
34                dt: float = 0.001) -> tuple[np.ndarray, np.ndarray]:
35        state0 = np.array([0.1, 0.0, -0.1, 0.0])
36        t_eval = np.arange(0, t_end, dt)
37        sol = solve_ivp(self.dynamics, [0, t_end], state0,
38                        t_eval=t_eval, method='RK45')
39
40        # Extract outputs
41        y1 = np.maximum(0, sol.y[0])
42        y2 = np.maximum(0, sol.y[2])
43        output = y1 - y2 # Net output
44
45        return sol.t, output
46

```

```

47
48 # Simulate and print summary
49 cpg = MatsuokaOscillator(tau1=0.1, tau2=0.3)
50 t, output = cpg.simulate(t_end=5.0)
51
52 # Estimate frequency from zero crossings
53 crossings = np.where(np.diff(np.sign(output)))[0]
54 if len(crossings) >= 4:
55     periods = np.diff(t[crossings[::2]])
56     freq = 1.0 / np.mean(periods)
57     print(f"CPG frequency: {freq:.2f} Hz")
58     print(f"CPG amplitude: {np.max(np.abs(output[len(output)//2:])):.3f}")

```

Listing 9.1: Matsuoka oscillator CPG implementation.

9.3 Phase Coupling and Inter-Limb Coordination

Multiple CPGs can be coupled to produce coordinated multi-limb movement:

$$\dot{\phi}_i = \omega_i + \sum_j K_{ij} \sin(\phi_j - \phi_i - \psi_{ij}), \quad (9.4)$$

where ϕ_i is the phase of the i -th oscillator, ω_i is its natural frequency, K_{ij} is the coupling strength, and ψ_{ij} is the desired phase relationship.

For quadruped locomotion:

- Walk: $\psi = (0, \pi, \pi/2, 3\pi/2)$ (diagonal pairs alternating)
- Trot: $\psi = (0, \pi, \pi, 0)$ (diagonal pairs synchronized)
- Gallop: $\psi = (0, 0, \pi, \pi)$ (front-back alternating)

Exercises

1. **CPG Tuning.** Vary τ_1/τ_2 ratio and plot the resulting frequency and duty cycle. How does the ratio control the waveform shape?
2. **Phase Coupling.** Implement 4 coupled oscillators and produce walk, trot, and gallop patterns by varying ψ_{ij} .
3. **(Programming.)** Use the Matsuoka CPG to drive a simulated bipedal walker. Tune the CPG parameters for stable walking.

Chapter 10

Motor Learning and Adaptation

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Practice does not make perfect.
Practice makes permanent. Only
perfect practice makes perfect.

10.1 Three Learning Systems

Motor learning involves three distinct neural systems with different timescales and mechanisms:

1. **Error-based learning** (cerebellum): uses sensory prediction errors (climbing fiber signals) to update cerebellar forward models via LTD [31, 24]. Trial-by-trial timescale (minutes to hours). It drives adaptation both to altered limb *dynamics* [38] and to an altered visuomotor *mapping* [25] — and those two are acquired independently of one another, so they are worth keeping distinct rather than filing together as “adaptation”.
2. **Reinforcement learning** (basal ganglia): uses dopamine reward signals to strengthen successful motor patterns [37]. Slower than error-based learning (hours to days). Enables discovery of novel solutions not implicit in the task structure.
3. **Consolidation and structural learning** (cortical): repetition of a movement biases future movements toward repeated patterns [9]. Timescale of hours to days; explains habit formation. Involves gradual changes in M1 tuning and synaptic weights.

10.2 Visuomotor Adaptation

The classic paradigm for studying error-based learning [25]:

```
1 import numpy as np
2
3 def simulate_visuomotor_adaptation(
4     n_baseline: int = 50,
```

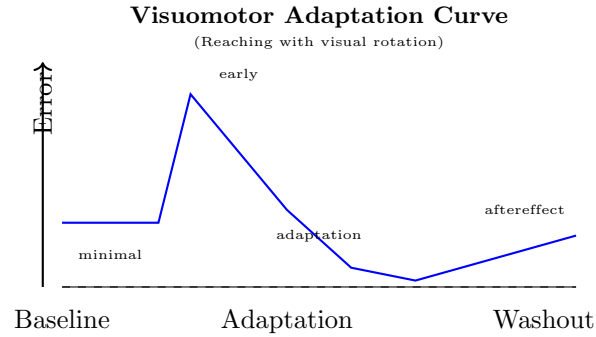


Figure 10.1: Characteristic visuomotor adaptation profile [25]: rapid error reduction (first 20 trials), gradual asymptote, persistent aftereffect demonstrating learning consolidation.

```

5     n_adaptation: int = 200,
6     n_washout: int = 100,
7     perturbation_deg: float = 30.0,
8     learning_rate: float = 0.05,
9     retention: float = 0.98
10 ) -> dict:
11     """Simulate visuomotor rotation adaptation.
12
13     A simple state-space model of adaptation:
14      $x_{k+1} = \text{retention} * x_k + \text{learning\_rate} * e_k$ 
15
16     Parameters
17     -----
18     n_baseline : int
19         Baseline trials (no perturbation).
20     n_adaptation : int
21         Adaptation trials (perturbation on).
22     n_washout : int
23         Post-adaptation trials (perturbation off).
24     perturbation_deg : float
25         Rotation perturbation in degrees.
26     learning_rate : float
27         Rate of error-based learning.
28     retention : float
29         Memory retention between trials.
30
31     Returns
32     -----
33     dict
34         Keys: 'trial', 'reaching_error', 'internal_state',
35              'phase' (labels for each trial)
36
37     """
38     n_total = n_baseline + n_adaptation + n_washout
39     p = np.radians(perturbation_deg)
40
41     # Internal state (learned compensation)
42     x = 0.0
43     errors = np.zeros(n_total)
44     states = np.zeros(n_total)
45     phases = []

```

```

46     for trial in range(n_total):
47         if trial < n_baseline:
48             perturbation = 0.0
49             phases.append('baseline')
50         elif trial < n_baseline + n_adaptation:
51             perturbation = p
52             phases.append('adaptation')
53         else:
54             perturbation = 0.0
55             phases.append('washout')
56
57         # Reaching error = perturbation - compensation
58         error = perturbation - x
59         errors[trial] = np.degrees(error)
60         states[trial] = np.degrees(x)
61
62         # Update internal state
63         x = retention * x + learning_rate * error
64
65     return {
66         'trial': np.arange(n_total),
67         'reaching_error': errors,
68         'internal_state': states,
69         'phase': phases
70     }
71
72
73 result = simulate Visuomotor adaptation()
74 print("Visuomotor Adaptation Summary:")
75 print(f" Baseline error: {np.mean(np.abs(result['reaching_error'][:50])):.1f} deg")
76 print(f" Final adaptation error: "
77       f"{np.abs(result['reaching_error'][-1]):.1f} deg")
78 print(f" Aftereffect: {result['reaching_error'][-1]:.1f} deg")

```

Listing 10.1: Visuomotor adaptation simulation.

10.3 Savings and Interference

10.3.1 Savings

Re-learning a previously learned adaptation is faster than initial learning, even after complete washout [20] — an observation that predates motor control, going back to Ebbinghaus's 1885 work on memory. This savings effect suggests that motor memories are not erased but suppressed, with additional learning circuits retaining implicit information about the learning dynamics.

10.3.2 Interference and Dual-Rate Learning

Learning opposite perturbations (A then B) produces anterograde interference: learning B slows retention of A (Li and Xie, 2000). This is explained by dual-rate learning models: a fast process learns trial-by-trial; a slow process consolidates. Interference occurs because the fast process is task-specific while the slow process is more general.

10.4 Structural Learning and Meta-Learning

Over many adaptation experiences, subjects develop the ability to adapt faster—they learn to learn (context-dependent learning). Smith et al. (1998) showed that prior exposure to one visuomotor rotation task accelerates learning of a novel rotation. This involves learning the structure of the task space itself, not just individual parameter values. Brain areas like dorsolateral prefrontal cortex appear to encode such meta-learning (Gershman et al., 2017).

Exercises

1. **State-Space Model.** Using the provided code, explore how learning rate and retention affect adaptation speed and aftereffect size.
2. **Dual-Rate Model.** Implement a dual-rate model (fast + slow process) and show that it can explain savings.
3. **(Programming.)** Simulate interference by adapting to $+30^\circ$ then -30° . Plot the adaptation curves and aftereffects.

Chapter 11

Computational Models of Motor Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

A theory of motor control must ultimately be a theory of computation: what is computed, how, and why.

Table 11.1: Comparison of Computational Motor Control Theories.

Theory	Learning Rule	Variability	Key Prediction
OFC (Todorov, 2004)	Error feedback	Tolerate task-irrelevant	Bell-shaped velocity profiles
Active Inference (Friston, 2010)	Minimize free energy	Prediction error-driven	Sensorimotor uncertainty shapes movement
Internal Model (Kawato, 1999)	Forward model learning (climbing fiber error)	Exploration + exploitation	Gradual visuomotor adaptation
Synergy Control (d’Avella, 2005)	Reduced-dim learning (NMF, ICA)	Synergy-space variability allowed	Movement flexibility without dimensionality

11.1 Optimal Feedback Control

The dominant computational theory of motor control is optimal feedback control (OFC), formulated by Todorov and Jordan [42]. OFC explains motor variability patterns (minimum intervention principle), predicts smooth velocity profiles, and unifies cerebellar learning with optimal control:

Definition 11.1. The OFC Framework

The nervous system controls movement by solving, in real-time, the stochastic optimal control problem:

$$\min_{\mathbf{u}(\cdot)} \mathbb{E} \left[\phi(\mathbf{x}_T) + \int_0^T \ell(\mathbf{x}_t, \mathbf{u}_t) dt \right], \quad (11.1)$$

subject to:

$$d\mathbf{x} = f(\mathbf{x}, \mathbf{u}) dt + G(\mathbf{x}) d\mathbf{w}, \quad (11.2)$$

where $d\mathbf{w}$ is Brownian noise representing signal-dependent motor noise.

Equation (11.1) is a *stochastic* optimal control problem, and the distinction matters. Because the dynamics carry a Brownian term, the solution is characterised by a Hamilton–Jacobi–Bellman equation for the value function, not by the Pontryagin maximum principle applied along a single trajectory. The deterministic geometric control theory this series draws on elsewhere—Agrachev & Sachkov [1]—treats smooth ordinary differential equations with no noise term and develops optimality through Pontryagin’s principle; it does not cover the problem above.

The reason a stochastic formulation is needed at all is that motor noise scales with the magnitude of the control signal, so it cannot be treated as a small additive disturbance around a nominal plan [16]. That single fact is what makes the minimum intervention principle below a prediction rather than a preference.

11.1.1 The Minimum Intervention Principle

OFC naturally explains why the nervous system tolerates task-irrelevant variability (the UCM from Chapter 1): the optimal controller only corrects deviations that affect the cost function. Task-irrelevant deviations are “free” and need not be corrected.

```

1 import numpy as np
2 from scipy.linalg import solve_continuous_are
3
4 def lqr_ofc(
5     A: np.ndarray,
6     B: np.ndarray,
7     Q: np.ndarray,
8     R: np.ndarray,
9     state_noise_cov: np.ndarray | None = None
10 ) -> tuple[np.ndarray, np.ndarray]:
11     """Compute LQR optimal feedback gains.
12
13     This implements the infinite-horizon OFC solution.
14
15     Parameters
16     -----
17     A : np.ndarray, shape (n, n)
18         State dynamics matrix.
19     B : np.ndarray, shape (n, m)
20         Input matrix.
21     Q : np.ndarray, shape (n, n)
22         State cost matrix.
23     R : np.ndarray, shape (m, m)

```

```

24         Control cost matrix.
25         state_noise_cov : np.ndarray, shape (n, n), optional
26             Process noise covariance.
27
28         Returns
29         -----
30         tuple
31             (K, P) where K is the optimal gain and P is the Riccati solution.
32         """
33         # Solve continuous algebraic Riccati equation
34         P = solve_continuous_are(A, B, Q, R)
35
36         # Optimal gain
37         K = np.linalg.solve(R, B.T @ P)
38
39         return K, P
40
41
42 # Example: 2D reaching with motor noise
43 # State: [x, y, dx, dy], Control: [Fx, Fy]
44 A = np.array([
45     [0, 0, 1, 0],
46     [0, 0, 0, 1],
47     [0, 0, -2, 0], # damping
48     [0, 0, 0, -2],
49 ])
50 B = np.array([
51     [0, 0],
52     [0, 0],
53     [1, 0],
54     [0, 1],
55 ])
56
57 # Task: reach to target with minimum effort
58 # High cost on position at target, low cost on path
59 Q = np.diag([10, 10, 1, 1]) # Position matters more than velocity
60 R = 0.01 * np.eye(2) # Low control cost
61
62 K, P = lqr_ofc(A, B, Q, R)
63 print("OFC gain matrix K:")
64 print(K)
65 print("\nMinimum intervention: gains are higher for")
66 print("task-relevant (position) than task-irrelevant dimensions")

```

Listing 11.1: Linear-quadratic optimal feedback controller.

11.2 Signal-Dependent Noise

A key feature of biological motor systems is that motor noise is **signal-dependent**: the noise variance scales with the control signal magnitude:

$$\mathbf{u} = \bar{\mathbf{u}} + \sigma(\bar{\mathbf{u}})\boldsymbol{\xi}, \quad \sigma(\bar{\mathbf{u}}) = k \|\bar{\mathbf{u}}\|, \quad (11.3)$$

where $\boldsymbol{\xi}$ is white Gaussian noise and k is the noise coefficient (empirically $k \approx 0.01$ – 0.05 in human reaching). This property has been measured extensively (Schmidt et al., 1979; Harris and Wolpert, 1998).

Under signal-dependent noise, the optimal trajectory minimizes integrated squared jerk (Flash and Hogan, 1985). This is a key prediction of OFC: the bell-shaped velocity profiles observed in human reaching emerge naturally as solutions to the signal-dependent noise minimization problem, not from any explicit minimization of jerk (Todorov, 2004).

11.3 Active Inference (Friston Framework)

Active inference (Friston, 2010; Friston et al., 2016) reframes motor control as inference: the agent minimizes prediction error by acting to make observations match its generative model. This is formalized through variational free energy minimization:

$$F = \mathbb{E}_q[\ln q(\mathbf{x}) - \ln p(\mathbf{x}, \mathbf{y})], \quad (11.4)$$

where F is the variational free energy, $q(\mathbf{x})$ is the approximate posterior belief, and $p(\mathbf{x}, \mathbf{y})$ is the generative model. The agent chooses actions to minimize F . This unifies action, perception, and learning under a single principle. While the experimental evidence is still developing, active inference correctly predicts many phenomena including the minimum intervention principle and why the brain is tolerant of task-irrelevant variability.

Exercises

1. **OFC Reaching.** Using the LQR code, simulate reaching movements to different targets. Show that task-irrelevant variability is tolerated.
2. **Signal-Dependent Noise.** Simulate reaching with and without signal-dependent noise. Plot the trajectory variability as a function of movement speed.
3. **(Programming.)** Implement the minimum-jerk model and the minimum-variance model. Show that both predict smooth bell-shaped velocity profiles.

Chapter 12

From Neural Architecture to Robot Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

If we truly understood how humans move, we could build robots that move like humans. We cannot. This tells us something.

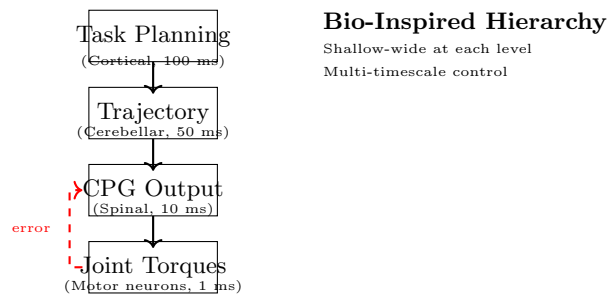


Figure 12.1: Hierarchical multi-timescale control architecture: each level shallow (1-2 layers) but wide (thousands of parallel channels), operating at increasing timescales and levels of abstraction.

12.1 Lessons From Biology for Robot Design

The four volumes of theory converge on practical design principles for robotic control systems:

Key Insight 12.1. Bio-Inspired Control Design Principles

1. **Exploit passive dynamics** (Vol II, Vol IV Ch 7): design the mechanical system so that natural dynamics do useful work. Use compliance, not rigidity.
2. **Use shallow-wide controllers** (Vol IV Ch 4): for real-time control, prefer 2–3 layer networks with thousands of neurons over deep networks with hundreds of layers.
3. **Employ synergy-based dimensionality reduction** (Vol IV Ch 4): define 4–8 actuation synergies and control in synergy space, not muscle space.
4. **Implement forward models** (Vol IV Ch 6): use the Smith predictor architecture to compensate for sensor/actuator delays.
5. **Design for contraction** (Vol I Ch 4): ensure the closed-loop system is contracting, guaranteeing stability and robustness.
6. **Optimize trajectories, not setpoints** (Vol II): design trajectory-centric controllers with funnel guarantees.
7. **Accept variability** (Vol IV Ch 1): allow task-irrelevant variability. Don't over-constrain the controller.

12.2 Cerebellar-Inspired Adaptive Control

The cerebellum's circuit architecture [31, 2, 24] directly suggests an adaptive control scheme:

- **Granule cells** (the bulk of some 69 billion cerebellar neurons [3]): create a compressed basis function representation of input state via mossy fiber–granule cell synapses
- **Purkinje cells** (~17 million): linearly combine granule cell outputs; synaptic weights change via LTD when climbing fiber error signals are present
- **Climbing fiber error** (inferior olive): carries sensory prediction error from brainstem nuclei; signals at ~1 Hz but with precise timing (10 ms resolution)

This maps directly to:

$$\mathbf{u}_{\text{adaptive}}(\mathbf{x}) = \hat{\mathbf{W}}^T \boldsymbol{\phi}(\mathbf{x}), \quad \dot{\hat{\mathbf{W}}} = -\Gamma \boldsymbol{\phi}(\mathbf{x}) \mathbf{e}^T, \quad (12.1)$$

where $\boldsymbol{\phi}(\mathbf{x})$ are radial basis functions (granule cell expansion), $\hat{\mathbf{W}}$ are adaptive weights (parallel fiber–Purkinje cell synapses), and $\mathbf{e}$ is the tracking error (climbing fiber signal).

12.3 CPG-Based Locomotion Controllers

Following Chapter 8, CPG-based locomotion controllers use coupled oscillators to generate walking/running patterns. This architecture is inspired by Grillner's [14] demonstration that deafferented spinal cords generate coordinated stepping. The CPG provides the *timing* and *synergy* structure; feedback provides *stability* and *responsiveness*:

```

1 import numpy as np
2
3 class BioInspiredLocomotionController:
4     """Hierarchical control architecture inspired by
5     biological motor control."""
6
7     def __init__(
8         self,
9         n_joints: int,
10        n_synergies: int = 4,
11        cpg_freq: float = 1.0
12    ):
13        self.n_joints = n_joints
14        self.n_synergies = n_synergies
15        self.cpg_freq = cpg_freq
16
17        # Synergy matrix (learned from motion data)
18        self.W = np.random.randn(n_joints, n_synergies) * 0.1
19
20        # CPG phases for each synergy
21        self.phases = np.linspace(
22            0, 2 * np.pi * (1 - 1/n_synergies), n_synergies
23        )
24
25        # Feedback gains (shallow: single layer)
26        self.K_fb = np.zeros((n_synergies, 2 * n_joints))
27
28    def cpg_output(self, t: float) -> np.ndarray:
29        """Generate rhythmic synergy activations from CPG."""
30        activations = np.array([
31            max(0.0, np.sin(2 * np.pi * self.cpg_freq * t + phi))
32            for phi in self.phases
33        ])
34        return activations
35
36    def feedforward(self, t: float) -> np.ndarray:
37        """Feedforward joint torques from CPG + synergies."""
38        c = self.cpg_output(t)
39        return self.W @ c
40
41    def feedback(self, state_error: np.ndarray) -> np.ndarray:
42        """Feedback correction (shallow: one layer)."""
43        c_fb = self.K_fb @ state_error
44        return self.W @ c_fb
45
46    def control(self, t: float,
47               state_error: np.ndarray) -> np.ndarray:
48        """Total control: feedforward (CPG) + feedback."""
49        return self.feedforward(t) + self.feedback(state_error)
50
51
52 # Example: 6-joint biped with 4 synergies
53 controller = BioInspiredLocomotionController(
54     n_joints=6, n_synergies=4, cpg_freq=1.2
55 )
56
57 # Generate one cycle of control signals
58 t = np.linspace(0, 1.0/1.2, 200)
59 torques = np.array([controller.feedforward(ti) for ti in t])
60

```

```

61 print(f"Bio-inspired controller: {controller.n_joints} joints, "
62       f"{controller.n_synergies} synergies")
63 print(f"Control dimensionality reduced from {controller.n_joints} "
64       f"to {controller.n_synergies}")
65 print(f"Peak torque magnitude: {np.max(np.abs(torques)):.3f}")

```

Listing 12.1: Bio-inspired locomotion controller architecture.

12.4 Whole-Body Control Architectures

For humanoid robots, the bio-inspired architecture suggests:

1. **Level 0 (Spinal)**: joint-level impedance control with reflex-like feedback (~ 1 ms loop)
2. **Level 1 (CPG)**: rhythmic pattern generation for locomotion (~ 10 ms loop)
3. **Level 2 (Cerebellar)**: adaptive feedforward based on forward model predictions (~ 50 ms loop)
4. **Level 3 (Cortical)**: trajectory planning and task-level control (~ 100 ms loop)

Each level is **shallow** (1–2 layers) but **wide** (many parallel channels), operating at progressively longer timescales.

12.5 Companion Software: Practical Implementation

Volume V provides the practical tools to implement and test these controllers using the pedagogical platform accompanying this textbook. The connection points:

- **MuJoCo**: test CPG-based locomotion controllers on musculoskeletal models
- **Drake**: implement trajectory optimization for trajectory-centric control
- **Pinocchio**: compute forward/inverse dynamics efficiently in real-time

Exercises

1. **Controller Architecture.** Using the bio-inspired controller code, vary the number of synergies and measure the resulting control signal dimensionality and smoothness.
2. **Cerebellar Adaptive Control.** Implement the cerebellar-inspired adaptive controller for a 2-DOF arm and show that it learns an inverse model over 50 trials.
3. **Hierarchical Design.** Design a 3-level hierarchical controller for a simulated biped, with impedance control at Level 0, CPG at Level 1, and trajectory correction at Level 2.
4. **(Programming.)** Implement the full bio-inspired controller for a MuJoCo walking model and benchmark against a standard PD controller.

Bibliography

- [1] Andrei A. Agrachev and Yuri L. Sachkov. *Control Theory from the Geometric Viewpoint*, volume 87 of *Encyclopaedia of Mathematical Sciences*. Springer, 2004.
- [2] J. S. Albus. A theory of cerebellar function. *Mathematical Biosciences*, 10(1-2):25–61, 1971.
- [3] Frederico A. C. Azevedo, Ludmila R. B. Carvalho, Lea T. Grinberg, José Marcelo Farfel, Renata E. L. Ferretti, Renata E. P. Leite, Wilson Jacob Filho, Roberto Lent, and Suzana Herculano-Houzel. Equal numbers of neuronal and nonneuronal cells make the human brain an isometrically scaled-up primate brain. *Journal of Comparative Neurology*, 513(5):532–541, 2009.
- [4] Richard Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961.
- [5] N. A. Bernstein. *The Coordination and Regulation of Movements*. Pergamon Press, 1967.
- [6] F. Bosch. *Strength Training and Coordination: An Integrative Approach*. On Target Publications, 2015.
- [7] Frans Bosch. *Anatomy of Agility: movement analysis in sport*. 20/10 Publishers, 2020.
- [8] Avis H. Cohen and Peter Wallén. The neuronal correlate of locomotion in fish: “fictive swimming” induced in an in vitro preparation of the lamprey spinal cord. *Experimental Brain Research*, 41(1), 1980.
- [9] J. D. Cohen, G. Aston-Jones, and M. S. Gilzenrat. A theory of how the basal ganglia generates and uses neural signals. *Trends in Neurosciences*, 28(11):574–586, 2005.
- [10] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.
- [11] Andrea d’Avella and Emilio Bizzi. Shared and specific muscle synergies in natural motor behaviors. *Proceedings of the National Academy of Sciences*, 102(8):3076–3081, 2005.
- [12] A. P. Georgopoulos, J. F. Kalaska, R. Caminiti, and J. T. Massey. On the relations between the direction of two-dimensional arm movements and cell discharge in primate motor cortex. *The Journal of Neuroscience*, 2(11):1527–1537, 1982.
- [13] Apostolos P. Georgopoulos, Andrew B. Schwartz, and Ronald E. Kettner. Neuronal population coding of movement direction. *Science*, 233(4771):1416–1419, 1986.
- [14] S. Grillner. Locomotion in vertebrates: central mechanisms and reflex interaction. *Physiological Reviews*, 55(2):247–304, 1975.
- [15] Hermann Haken, J. A. Scott Kelso, and H. Bunz. A theoretical model of phase transitions in human hand movements. *Biological Cybernetics*, 51(5):347–356, 1985.

- [16] Christopher M. Harris and Daniel M. Wolpert. Signal-dependent noise determines motor planning. *Nature*, 394(6695):780–784, 1998.
- [17] A. V. Hill. The heat of shortening and the dynamic constants of muscle. *Proceedings of the Royal Society B: Biological Sciences*, 126(843):136–195, 1938.
- [18] Neville Hogan. Adaptive control of mechanical impedance by coactivation of antagonist muscles. *IEEE Transactions on Automatic Control*, 29(8):681–690, 1984.
- [19] B. Hommel, J. Müsseler, G. Aschersleben, and W. Prinz. The theory of event coding (tec): a framework for perception and action planning. *Behavioral and Brain Sciences*, 24(5):849–878, 2001.
- [20] Vincent S. Huang, Adrian Haith, Pietro Mazzoni, and John W. Krakauer. Rethinking motor learning and savings in adaptation paradigms: model-free memory for successful actions combines with internal models. *Neuron*, 70(4):787–801, 2011.
- [21] Hiroshi Imamizu, Satoru Miyauchi, Tomoe Tamada, Yuka Sasaki, Ryousuke Takino, Benno Pütz, Toshinori Yoshioka, and Mitsuo Kawato. Human cerebellar activity reflecting an acquired internal model of a new tool. *Nature*, 403(6766):192–195, 2000.
- [22] W. James. *The Principles of Psychology*. Holt, Rinehart and Winston, 1890.
- [23] Bernard Katz and Ricardo Miledi. The measurement of synaptic delay, and the time course of acetylcholine release at the neuromuscular junction. *Proceedings of the Royal Society of London. Series B. Biological Sciences*, 161(985):483–495, 1965.
- [24] M. Kawato, K. Furukawa, and R. Suzuki. A hierarchical neural-network model for control and learning of voluntary movement. *Biological Cybernetics*, 57(3):169–185, 1987.
- [25] John W. Krakauer, Maria-Felice Ghilardi, and Claude Ghez. Independent learning of internal models for kinematic and dynamic control of reaching. *Nature Neuroscience*, 2(11):1026–1031, 1999.
- [26] Jason J. Kutch and Francisco J. Valero-Cuevas. Challenges and new approaches to proving the existence of muscle synergies of neural origin. *PLoS Computational Biology*, 8(5):e1002434, 2012.
- [27] Mark L. Latash. The bliss (not the problem) of motor abundance (not redundancy). *Experimental Brain Research*, 217(1):1–5, 2012.
- [28] Daniel D. Lee and H. Sebastian Seung. Algorithms for non-negative matrix factorization. In *Advances in Neural Information Processing Systems*, volume 13, 2000.
- [29] G. E. Loeb. Control implications of musculoskeletal mechanics. In *Proceedings of the 17th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, volume 2, pages 1393–1394, Montreal, Canada, 1995.
- [30] Ian D. Loram and Martin Lakie. Direct measurement of human ankle stiffness during quiet standing: The intrinsic mechanical stiffness is insufficient for stability. *The Journal of Physiology*, 545(3):1041–1053, 2002.

- [31] D. Marr. A theory of cerebellar cortex. *Journal of Physiology*, 202(2):437–470, 1969.
- [32] K. Matsuoka. Sustained oscillations generated by mutually inhibiting neurons with adaptation. *Biological Cybernetics*, 52(6):367–376, 1985.
- [33] Matthew Millard, Thomas Uchida, Ajay Seth, and Scott L. Delp. Flexing computational muscle: Modeling and simulation of musculotendon dynamics. *Journal of Biomechanical Engineering*, 135(2):021005, 2013.
- [34] Karl M. Newell. Constraints on the development of coordination. In M. G. Wade and H. T. A. Whiting, editors, *Motor Development in Children: Aspects of Coordination and Control*, pages 341–360. Martinus Nijhoff, 1986.
- [35] Wilder Penfield and Edwin Boldrey. Somatic motor and sensory representation in the cerebral cortex of man as studied by electrical stimulation. *Brain*, 60(4):389–443, 1937.
- [36] J. P. Scholz and G. Schöner. The uncontrolled manifold concept: Identifying control variables for a functional task. *Experimental Brain Research*, 126:289–306, 1999.
- [37] W. Schultz, P. Dayan, and P. R. Montague. A neural substrate of prediction and reward. *Science*, 275(5307):1593–1599, 1997.
- [38] R. Shadmehr and F. A. Mussa-Ivaldi. Adaptive representation of dynamics during learning of a motor task. *Journal of Neuroscience*, 14(5):3208–3224, 1994.
- [39] Maurice A. Smith and Reza Shadmehr. Intact ability to learn internal models of arm dynamics in huntington’s disease but not cerebellar degeneration. *Journal of Neurophysiology*, 93(5):2809–2821, 2005.
- [40] Michele M. Solis and David J. Perkel. Rhythmic activity in a forebrain vocal control nucleus in vitro. *The Journal of Neuroscience*, 25(11):2811–2822, 2005.
- [41] Aparna Suvrathan, Hannah L. Payne, and Jennifer L. Raymond. Timing rules for synaptic plasticity matched to behavioral function. *Neuron*, 92(5):959–967, 2016.
- [42] E. Todorov and M. I. Jordan. Optimal feedback control as a theory of motor coordination. *Nature Neuroscience*, 5(11):1226–1235, 2002.
- [43] M. C. Tresch, V. C. K. Cheung, and A. d’Avella. Matrix factorization algorithms for the identification of muscle synergies: Evaluation on simulated and experimental data sets. *Journal of Neurophysiology*, 95(4):2199–2212, 2006.
- [44] D. M. Wolpert and M. Kawato. Multiple paired forward and inverse models for motor control. *Neural Networks*, 11(7-8):1317–1329, 1998.
- [45] Naoki Yamawaki, Katharine Borges, Benjamin A. Suter, Kenneth D. Harris, and Gordon M. G. Shepherd. A genuine layer 4 in motor cortex with prototypical synaptic circuit connectivity. *eLife*, 3:e05422, 2014.